

Bangladesh University of Engineering and Technology

Department of Computer Science & Engineering

Structured Programming Question Bank

CSE 101 · Structured Programming Language

Year Coverage: 2003–04 to 2024–25

Topics: Data Types & Operators, Control Structures, Functions,
Arrays & Strings, Recursion, Pointers, Structures & Unions,
File I/O, Bitwise Operations, Preprocessors, Error Handling

Authors & Contributors

Md Ratul Hasan (2405009) — Question Curation & Organisation
— $\text{\LaTeX}$ Typesetting
NotebookLM — Research & Extraction
Claude AI — $\text{\LaTeX}$ Typesetting
Gemini — Diagram Generation

Table of Contents

Part I — Fundamentals

- ▷ **S1.** Data Types, Variables & Operators
 - Data Types & Variables
 - Operators, Expressions & Type-Casting
- ▷ **S2.** Control Structures
 - if-else & switch-case
 - Loops (while, do-while, for)
 - Nested Control, break & continue
 - goto Statement

Part II — Functions & Arrays

- ▷ **S3.** Functions
 - Function Basics & Parameter Passing
 - Recursion
- ▷ **S4.** Arrays & Strings
 - One-Dimensional Arrays: Searching & Sorting
 - Character & String Operations
 - Multi-Dimensional Arrays & Matrix Operations

Part III — Pointers & User-Defined Types

- ▷ **S5.** Pointers
 - Pointer Basics
 - Pointers to Arrays, Strings, Structs & Functions
 - Dynamic Memory Allocation
- ▷ **S6.** User-Defined Data Types
 - Structures
 - Unions, Bit-fields & Enumerations

Part IV — I/O, Bitwise, Preprocessors & Miscellaneous

- ▷ **S7.** Input/Output
 - Console & Formatted I/O
 - File I/O & Command Line Arguments
- ▷ **S8.** Bitwise Operations
 - Bitwise Operators & Applications
- ▷ **S9.** Preprocessors, Var-Arg & Error Handling
 - Header Files & Preprocessors
 - Variable Argument Functions
 - Error Handling

Part V — Data Structures

▷ **S10.** Abstract Data Structures

· Linked Lists

· Stacks & Queues

· Binary Search Trees

BUET · CSE 101 · Structured Programming

Part I

Fundamentals

Fundamentals

S1 — Data Types, Variables & Operators

Data Types & Variables

- Q1.** You want to store the vaccination records of diphtheria, typhoid, cholera, polio, tetanus, and measles applied to a child (each vaccination is either given or not). Which data type will be optimal for storing this information? Justify your answer. **(4 marks)**
[CSE 101 | L-1/T-1 | 2024–25, 2023–24]
- Q2.** Write down the desirable characteristics of a good program. Define escape sequences with examples. **(9 marks)** [CSE 101 | L-1/T-1 | 2020–21]
- Q3.** What is meant by a *token*? Describe why the distinction between a reserved word and a keyword is significant in natural language but not in programming language. **(7 marks)** [CSE 101 | L-1/T-1 | 2020–21]
- Q4.** How is a static local variable declared? Explain with an example how a static local variable differs from a regular local variable. **(10 marks)** [CSE 101 | L-1/T-1 | 2018–19]
- Q5.** Write a C program that will display the details of the program's running configuration environment on the monitor screen. **(7 marks)** [CSE 105 | L-1/T-2 | 2013–14]
- Q6.** Consider the following code fragment:

```
1 int i = 'c';
```

Are we allowed to assign a character constant to an `int` variable? First answer yes or no, then explain your reasoning. **(5 marks)**
[CSE 105 | L-1/T-2 | 2012–13]

- Q7.** What is an escape sequence? What is the purpose of using escape sequences in C programming? **(5 marks)** [CSE 105 | L-1/T-2 | 2011–12]
- Q8.** Write down some characteristics that are to be followed when writing a good computer program. **(6 marks)** [CSE 105 | L-1/T-2 | 2011–12]
- Q9.** Write some differences between `signed` and `unsigned` data types. Can these qualifiers be applied to `double` or `float`? **(5 marks)**
[CSE 105 | L-1/T-2 | 2010–11]
- Q10.** Write a simple program to find the size of different basic data types in C. **(5 marks)** [CSE 105 | L-1/T-2 | 2010–11]
- Q11.** Write short notes on local, global, and static variables. What are the features of static local and static global variables? **(10 marks)**
[CSE 105 | L-1/T-2 | 2010–11]
- Q12.** What will be the output of the following program? Explain your answer.

```
1 #include <stdio.h>
2 int main() {
3     short int nl = 130;
4     char ch = 'a';
5     printf("\n nl's value is: %d", nl);
6     printf("\n ch's value is: %d", ch);
7     nl += ch + 230;
8     ch += nl + 470;
9     printf("\n nl's value is: %d", nl);
10    printf("\n ch's value is: %c", ch);
11    printf("\n");
12    return 0;
13 }
```

(10 marks)

[CSE 101 | L-1/T-2 | 2009–10]

- Q13.** Which of the following variable declarations is right or wrong, and why?

```
1 char ch = 256;
2 int _;
3 int long il;
4 char [25]name;
5 char *cp = 'z';
6 const int n;
7 long l;
8 double average.cgpa;
```

```

9  long float lf;
10 short char shortch;
    
```

(15 marks)

[CSE 101 | L-1/T-2 | 2009–10]

- Q14.** When you want to manipulate a variable in a program by several functions, you can either (a) define the variable as a global variable with functions having no parameters and void return type, or (b) define the variable as a local variable with functions having appropriate parameters and return types. Write about the pros and cons of each approach. (10 marks) [CSE 101 | L-1/T-2 | 2009–10]
- Q15.** With appropriate examples, describe different types of storage class specifications used in a C program. (12 marks) [CSE 101 | L-1/T-2 | 2008–09]
- Q16.** Present a table showing the range and size of all primitive datatypes in C which can be used to store non-fractional numeric values. (10 marks) [CSE 105 | L-1/T-2 | 2007–08]
- Q17.** What do you mean by the scope of an identifier? Discuss the scopes for a global variable. (7 marks) [CSE 105 | L-1/T-2 | 2005–06]
- Q18.** Discuss different types of storage class specifiers. (10 marks) [CSE 105 | L-1/T-2 | 2005–06]
- Q19.** We know that $5 = (101)_2$ and $7 = (111)_2$, while $10 = (1010)_2$ and $6 = (110)_2$. It is interesting to see that in the binary representation of an odd number the LSB (least significant, i.e. rightmost bit) is 1, and for an even number it is 0. Prove that this holds for any odd or even number. (12 marks) [CSE 101N | L-1/T-1 | 2003–04]
- Q20.** An integer (2 bytes) has the value 20 in its low byte and 17 in its high byte. What is the value of the integer in decimal and hexadecimal? (6 marks) [CSE 101N | L-1/T-1 | 2003–04]

Operators, Expressions & Type-Casting

- Q1.** Write the equivalent expression of the followings without using any relational or logical operator:

```

1  int a, b, c, d;
2  scanf("%d%d%d%d", &a, &b, &c, &d);
3  if(a >= b && c >= d)
4      printf("Hi");
5  else
6      printf("Bye");
    
```

(5 marks)

[CSE 101 | L-1/T-1 | 2024–25]

- Q2.** Suppose you are given an integer n and a digit d . Build another integer r where n is reversed first and then d is inserted in front of it. For example, if $n = 532$ and $d = 8$, then $r = 8235$. Write the complete C program. (10 marks) [CSE 101 | L-1/T-1 | 2022–23]

- Q3.** Determine the output of the following C code segment:

```

1  int x=20, y=10, z=0;
2  int a = x && !y || z;
3  int b = x++ * --y + !z;
4  printf("%d %d\n", a, b);
    
```

(5 marks)

[CSE 101 | L-1/T-1 | 2022–23]

- Q4.** Write down the following three functions with the specified tasks. Assume that always a negative value will be supplied in parameter x . You are **not** allowed to use any branching or repetition statements.
- (a) `int truncate(float x)` — discards the fractional part and returns the integer.
 - (b) `int rounding(float x)` — returns the nearest integer; in case of a tie, returns the larger one.
 - (c) `int ceiling(float x)` — returns the smallest integer not less than x .

(15 marks)

[CSE 101 | L-1/T-1 | 2022–23]

- Q5.** Write two functions that return the ceiling and floor of a fractional number up to n decimal places. The functions `ceiling` and `floor` return the nearest fractional numbers with n decimal places greater than or equal to, and smaller than or equal to, the argument respectively. The use of any built-in function is not allowed. (10 marks) [CSE 101 | L-1/T-1 | 2021–22]

- Q6.** Consider the following program codes:


```

1 int i = 8, j = 5, k;
2 float x = 0.005, y = -0.01;
3 char c = 'c', d = 'd';
4 k = (i - 3 * j) % (c + 2 * d) / (x - y);

```

Rewrite the last line using explicit type casting to avoid any error or warning. Evaluate the expression to find the value of **k**. How many times does type auto-conversion take place while evaluating the equation? **(9 marks)** [CSE 101 | L-1/T-1 | 2020–21]

Q7. Suppose you are given an integer **n** and a digit **d**. Write a program to build another integer **r** where **d** is inserted in front of **n**. For example, if **n** = 532 and **d** = 8, then **r** will be 8532. **(10 marks)** [CSE 101 | L-1/T-1 | 2019–20]

Q8. Write the following two functions (assume a negative value will always be supplied):

- (a) `int truncate(float x)` — discards the fractional part and returns the integer.
- (b) `int rounding(float x)` — returns the nearest integer (in case of tie, returns the larger one).
- (c) `int ceiling(float x)` — returns the smallest integer not less than **x**.

You are not allowed to use any branching or repetition statements. **(15 marks)** [CSE 101 | L-1/T-1 | 2019–20]

Q9. What will be the output of the following program, if the input is 0?

```

1 #include <stdio.h>
2 int main() {
3     int i;
4     scanf("%d", &i);
5     if (++i) {
6         int i = 50;
7         printf("%d\n", i++);
8     }
9     printf("%d\n", i);
10    return 0;
11 }

```

(5 marks) [CSE 101 | L-1/T-1 | 2018–19]

Q10. Is the following code fragment valid? What will be the output if so? State your reason.

```

1 int a = 2, b = 5, c;
2 c = a +++ b;
3 printf("%d, %d, %d", a, b, c);

```

(5 marks) [CSE 101 | L-1/T-1 | 2015–16]

Q11. Write a program that takes an English uppercase letter as input and prints the letter two positions after it in the alphabet in lowercase. For example, if the user enters **D** the output is **f**; if the input is **Z** the output is **b**. You are not allowed to use **if-else**, **switch-case**, or the ternary operator. **(10 marks)** [CSE 101 | L-1/T-1 | 2015–16]

Q12. Suppose you are working with an integer (4 bytes, 32 bits). How can you find the value of the rightmost byte (8 bits) without using any bitwise operator? **(5 marks)** [CSE 101 | L-1/T-1 | 2015–16]

Q13. State whether the following relational expressions are true or false:

- (a) `'A' - 'Z' == 'a' - 'z'`
- (b) `'0' == 0`
- (c) `(x > 0) && (x <= 0)`
- (d) `~(x (~x)) == 0`

(6 marks) [CSE 105 | L-1/T-2 | 2014–15]

Q14. The modulus operator (%) finds the remainder when integer **x** is divided by integer **y**. Show how to find the remainder without using the modulus operator (you may use any other mathematical operators as many times as needed). **(6 marks)** [CSE 105 | L-1/T-2 | 2014–15]

Q15. You are given an unsigned integer as input. Using its bit pattern to represent a floating-point number, produce the corresponding floating-point value. Write the necessary code and explain. **(5 marks)** [CSE 105 | L-1/T-2 | 2014–15]

Q16. Assume `int x` has been declared and initialized to an unknown value. For each of the following, indicate if the expression **always** evaluates to true, or if it could **sometimes** be false (and if so, give a counterexample value of **x**):

- (a) `(x != 0) || (x == 0)`
- (b) `(x > 0) || (x < 0)`

(c) `((x >> 1) << 1) | (x & 1)) == x`

(d) `(x ^ (~x)) == 0`

(6 marks)

[CSE 105 | L-1/T-2 | 2013–14]

Q17. The ceiling of a fractional number is the smallest integer greater than or equal to it (e.g., ceiling of 3.12 is 4; ceiling of -3.12 is -3). Write a C program that takes a float as input and prints its ceiling. You are not allowed to use any math library function. (7 marks)

[CSE 105 | L-1/T-2 | 2013–14]

Q18. What is the output of the following code segment (write NO OUTPUT if there is an error)?

```
1 int i = 4, j = 5, k;
2 k = i +++ j;
3 printf("%d %d %d", i, j, k);
```

(5 marks)

[CSE 105 | L-1/T-2 | 2013–14]

Q19. The ceiling of a float value is the smallest integer greater than or equal to it; the flooring is the largest integer smaller than or equal to it (e.g., ceiling of 3.12 is 4, flooring is 3). Write a C program that takes a float value as input and prints its ceiling and flooring values. You are not allowed to use any library function except I/O-related ones. (Hint: Think about type conversion.) (10 marks)

[CSE 105 | L-1/T-2 | 2012–13]

Q20. Without using `if/else` or the `?` operator, write a function `int PositiveSubtract(int x, int y)` which computes $x - y$ when $x \geq y$, and returns 0 otherwise. (5 marks)

[CSE 105 | L-1/T-2 | 2010–11]

Q21. Describe the use of the conditional operator to form conditional expressions. How is the following statement evaluated if `b = 7`?

```
1 x = b > 8 ? b << 3 : b > 4 ? b >> 1 : b;
```

(7 marks)

[CSE 101 | L-1/T-2 | 2008–09]

Q22. Explain why the comparison `-1L > 1UL` will be evaluated as true in C. (5 marks)

[CSE 105 | L-1/T-2 | 2007–08]

Q23. What will be the output of the following statement? Explain your answer.

```
1 int var = 5;
2 printf("%d %d %d %d", var, ++var, var++, var);
```

(8 marks)

[CSE 105 | L-1/T-2 | 2005–06]

Q24. What are precedence and associativity of operators? In C, there are some operators that can take different numbers of operands. What are they? (8 marks)

[CSE 101N | L-1/T-1 | 2003–04]

Q25. A programmer was asked to write a program to find the surface area of a sphere given its diameter. The following code compiles and runs, but does not work correctly. List **all** the problems:

```
1 #include <stdio.h>
2 #define PI 3.14
3 int main() {
4     float d, area;
5     int r;
6     printf("what is the diameter of the sphere?");
7     scanf("%f", &d);
8     r = d * (1/2);
9     area = 4 * PI * (r * r);
10    printf("Surface area = %f", area);
11    return 0;
12 }
```

(5 marks)

[CSE 101N | L-1/T-1 | 2003–04]

S2 — Control Structures

if-else & switch-case

Q1. Write down a program that will take a year as input and will print whether the year is a leap year or not. You are only allowed to use switch-case statement to solve the problem (you cannot use if-else or any ternary operator) (15 marks) [CSE 101 | L-1/T-1 | 2024–25]

Q2. Write down a program that will take four integers as input and will print the second largest of the four. **(15 marks)** [CSE 101 | L-1/T-1 | 2024–25]

Q3. Write a C program segment that takes an integer n as input and prints YES if n is divisible by 2 **or** by 5, and prints NO otherwise. Use **only a single if-else**. You are **not** allowed to use any logical operator or any nested branching. **(5 marks)** [CSE 101 | L-1/T-1 | 2023–24]

Q4. The speed limit on a highway is 100 km/h. If someone is caught speeding, they receive a fine that depends on their speed and whether they have had a previous speeding ticket, according to the table below:

Speed (km/h)	Previous ticket?	Fine
≤ 100	N/A	\$0
101–120	no	\$50
> 120	no	\$80
> 100	yes	\$150

Write a program that reads an integer speed (km/h) and a y/n response for a previous ticket, then outputs the fine amount. Properly organise all branches of **if-else if** so they are clearly visible and readable. **(10 marks)** [CSE 101 | L-1/T-1 | 2022–23]

Q5. Write a program that takes an integer as input and prints YES if the number is divisible by 2 but **not** divisible by 5, and prints NO otherwise. You **must** solve this using a **switch-case** statement. **(7 marks)** [CSE 101 | L-1/T-1 | 2022–23]

Q6. Give equivalent logical expressions for the following **without** using negation:

- (a) $!(a > b)$
- (b) $!(a \leq b \ \&\& \ c \leq d)$
- (c) $!(a + 1 == b + 1)$
- (d) $!(a < 1 \ || \ b < 2 \ \&\& \ c < 3)$

(6 marks)

[CSE 101 | L-1/T-1 | 2019–20]

Q7. Convert the following **if-else** statement into a **switch-case** statement (nested **switch-case** allowed):

```
1 if (a > b && a > c)
2     printf("a is the boss");
3 else
4     printf("a is not the boss");
```

(7 marks)

[CSE 101 | L-1/T-1 | 2019–20]

Q8. Write a program that detects whether a point is inside or outside a rectangle whose sides are parallel to the XY axes. Input: (x, y) coordinates of 3 integer points — the first two are the top-left and bottom-right corners of the rectangle, the third is the query point. Print "Inside the rectangle" if the query point is inside or on the boundary, otherwise print "Outside the rectangle". **(10 marks)** [CSE 101 | L-1/T-1 | 2018–19]

Q9. For the first 100 units the price is 1.0 Taka/unit; for the next 150 units it is 1.25 Taka/unit; beyond that it is 1.5 Taka/unit. Write a program that takes an integer n (units bought) and outputs the total price to 2 decimal places. **(10 marks)** [CSE 101 | L-1/T-1 | 2018–19]

Q10. Write a program that takes a character as input and prints one of the following as appropriate: Vowel in capital letter, Vowel in small letter, Consonant in capital letter, Consonant in small letter, Neither a vowel nor a consonant. **(10 marks)** [CSE 101 | L-1/T-1 | 2018–19]

Q11. Write a short function `lwrcase(ch)` which accepts a character parameter `ch` and returns the equivalent lowercase letter only if `ch` is an uppercase letter; otherwise it returns the same character unchanged. Use only **switch-case** (no **if-else**, no library functions). **(8 marks)** [CSE 101 | L-1/T-1 | 2015–16]

Q12. A year y is a leap year if: (i) y is divisible by 400, or (ii) y is divisible by 4 but not by 100. Fill in the condition marked ? to correctly test for a non-leap year:

```
1 if ( ? )
2     printf("LOL! Not a leap year");
3 else
4     printf("YAHOO! Leap year");
```

(7 marks)

[CSE 101 | L-1/T-1 | 2015–16]

Q13. Show a code segment to determine the largest of three fractional numbers a , b , and c . You are only allowed to use the conditional operator (`?:`) as many times as needed (no **if-else**, no **switch-case**). **(5 marks)** [CSE 101 | L-1/T-1 | 2015–16]

Q14. Write a short function `upcase(ch)` which accepts a character parameter `ch` and returns the same character unless it is a lowercase ASCII character, in which case it returns the equivalent uppercase character. You may only use a `switch-case` statement (no `if-else`, no library functions). (8 marks) [CSE 105 | L-1/T-2 | 2014–15]

Q15. Misir Ali wrote the following code to find the largest of three integers `a`, `b`, `c`:

```
1 if (a > b > c) max = a;
2 else if (b > a > c) max = b;
3 else max = c;
```

Is he a good programmer? What will `max` be if `a=20`, `b=10`, `c=5`? Write a single-line statement using the conditional operator (`?:`) to find the maximum of three numbers `a`, `b`, `c`. (10 marks) [CSE 105 | L-1/T-2 | 2013–14]

Q16. The speed limit on a highway is 100 km/h. Speeding fines depend on speed and whether the driver has a previous ticket:

Speed (km/h)	Previous ticket?	Fine
100 or less	N/A	\$0
101–120	no	\$50
>120	no	\$80
>100	yes	\$150

Write a program that asks for an integer speed (km/h) and a y/n response for a previous ticket, then outputs the fine amount (\$0, \$50, \$80, or \$150). (10 marks) [CSE 105 | L-1/T-2 | 2012–13]

Q17. When we say `a = b`, `a` is assigned the value of `b` and the expression evaluates to the value of `a` after the assignment. Carefully explain when the `printf` in the following code would execute:

```
1 if (a = b) printf("The if statement is executing.");
```

(4 marks)

[CSE 105 | L-1/T-2 | 2012–13]

Q18. Briefly describe what is meant by *short-circuit evaluation*, and give a code fragment whose behaviour depends on it. (6 marks) [CSE 105 | L-1/T-2 | 2012–13]

Q19. State two limitations of a `switch` statement compared to an `if-else` statement, with examples. (5 marks) [CSE 105 | L-1/T-2 | 2012–13]

Q20. Explain what happens when the following statement is executed:

```
1 if (abs(x) < xmin)
2     x = (x > 0) ? xmin : -xmin;
```

Is this a compound statement? Is a compound statement embedded within this statement? (9 marks) [CSE 105 | L-1/T-2 | 2011–12]

Q21. Explain what will be the output of the following code:

```
1 #include <stdio.h>
2 void main() {
3     int k = 10;
4     switch (k % 2) {
5         case 0: k += 2;
6         case 1: k = 0;
7         case 5: k = 1;
8     };
9     printf("k= %d", k);
10 }
```

(5 marks)

[CSE 105 | L-1/T-2 | 2010–11]

Q22. Suppose you have to write a simple program that will convert marks into letter grades. You can assume that all marks will be within 60 and 100.

Marks	Grade
80+	A+
75–79	A
70–74	A-
65–69	B+
60–64	B

Write a simple, efficient program that will take marks as inputs and output the corresponding letter grades using `switch/case`. (10 marks) [CSE 105 | L-1/T-2 | 2005–06]

- Q23.** Why should we use `break` after each `case` in a `switch` statement? What happens if it is omitted? (5 marks) [CSE 101N | L-1/T-1 | 2003–04]
- Q24.** What is the “dangling `else`” problem? How does ambiguity arise and how can it be resolved? (5 marks) [CSE 101N | L-1/T-1 | 2003–04]
- Q25.** Consider the following code:

```

1 x = a;
2 if (x > a)
3     x = 6;
4 else
5     x++;

```

What is the minimum equivalent code that has the same effect on `x`? (4 marks) [CSE 101N | L-1/T-1 | 2003–04]

Loops: while, do-while, for

- Q1.** In number theory, a semi-prime is a natural number that is the product of two prime numbers. For example, the semi-primes less than 100 are:
4, 6, 9, 10, 14, 15, 21, 22, 25, 26, 33, 34, 35, 38, 39, 46, 49, 51, 55, 57, 58, 62, 65, 69, 74, 77, 82, 85, 86, 87, 91, 93, 94, and 95.
Given a positive integer n as input, find whether a number is a semiprime or not. Print *True* if number is semi-prime, print *False*. (15 marks) [CSE 101 | L-1/T-1 | 2024–25]
- Q2.** Given an integer N as input, write down a program that will print how many times each digit (0 to 9) appeared in the number.
- Sample Input**
N: 223
- Sample Output**
2 appeared 2 times
3 appeared 1 times (15 marks) [CSE 101 | L-1/T-1 | 2024–25]
- Q3.** In number theory, a prime g is a **Sophie Germain prime** if $2g + 1$ is also prime. The Sophie Germain primes less than 100 are: 2, 3, 5, 11, 23, 29, 41, 53, 83, 89. Write a program that takes an integer K as input and prints all Sophie Germain primes less than or equal to K . (20 marks) [CSE 101 | L-1/T-1 | 2023–24]
- Q4.** A Mersenne number is a positive integer of the form $M_n = 2^n - 1$. A **Mersenne prime** is a Mersenne number that is prime (e.g. $M_2 = 3$ is the smallest). Design and implement a complete C program that finds and prints the **five smallest** Mersenne primes, each on its own line. You may use any C math library functions. (18/15 marks) [CSE 101 | L-1/T-1 | 2022–23] [CSE 105 | L-1/T-2 | 2012–13]
- Q5.** Test the following code fragment. What will be the output if `j` is set to the last digit of your student ID?

```

1 int a[10];
2 int i, j;
3 j = last_digit_of_your_studentID;
4 for (i=0; i<10; i++)
5     a[i] = 5-i;
6 printf("%d\n", a[j]);

```

Show your calculation. (7 marks) [CSE 101 | L-1/T-1 | 2022–23]

- Q6.** Write a program to compute the summation of the series $a_n x^n + a_{n-1} x^{n-1} + \dots + a_0 x^0$. Use the minimum number of loops in your program. (10 marks) [CSE 101 | L-1/T-1 | 2021–22]
- Q7.** Write a program that determines whether an input integer x (between 1 and 100) is prime. Use the minimum number of division operations and explain the correctness of the method used. (10 marks) [CSE 101 | L-1/T-1 | 2021–22]
- Q8.** A palindromic number remains the same when its digits are reversed (e.g. 984489 and 12321 are palindromes; 1234 and 985489 are not). Write a C function `int isPalindrome(int x)` that returns 1 if `x` is a palindrome and 0 otherwise. Do not use any array or pointer. (10 marks) [CSE 101 | L-1/T-1 | 2020–21]
- Q9.** The sine of x can be calculated approximately by summing the first n terms of the series:

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

where x is in radians (π rad = 180). Write a C function `double sin(double x)` that takes the angle in degrees and returns $\sin x$ accurate to 3 decimal places. Do not use any math library functions. (10 marks) [CSE 101 | L-1/T-1 | 2020–21]

Q10. Starting from integer x , print n odd numbers. Both n and x will be input to your program. (7 marks) [CSE 101 | L-1/T-1 | 2019–20]

Q11. A **semiprime** is a natural number that is the product of exactly two prime numbers (e.g., 4, 6, 9, 10, ...). Write a program that takes an integer N as input and determines whether N is semiprime or not. (10 marks) [CSE 101 | L-1/T-1 | 2018–19]

Q12. The Fibonacci series starts 1, 1, 2, 3, 5, 8, 13, ... where each number is the sum of the two preceding ones. Write a program that takes $N > 0$ as input and:

(a) Checks whether N belongs to the Fibonacci series and prints YES or NO.

(b) Finds and prints the sum of all Fibonacci numbers $\leq N$.

(10 marks)

[CSE 101 | L-1/T-1 | 2018–19]

Q13. Write a code segment that finds and prints the sum of the first n odd numbers (without using any mathematical formula). n will be input to your program. (7 marks) [CSE 101 | L-1/T-1 | 2015–16]

Q14. Write a program that takes an integer n and a digit d as input and determines how many times the digit d appears in n . (10 marks) [CSE 101 | L-1/T-1 | 2015–16]

Q15. Write a program that asks for values of two integers a and n and computes:

$$a^2 - (a-1)^2 + (a-2)^2 - (a-3)^2 + \dots \pm (a-n)^2$$

You are not allowed to use any library functions except I/O-related ones. (10 marks)

[CSE 101 | L-1/T-1 | 2015–16]

Q16. Write a function `checkInTheSeries(x)` which returns 1 if x belongs to the series defined by $f(0) = 1$, $f(n) = n \times f(n-1) + n$ (first 6 values: 1, 2, 6, 21, 88, 445), and 0 otherwise. (11 marks) [CSE 101 | L-1/T-1 | 2015–16]

Q17. Show a program fragment that will repeatedly read two numbers A and B until both are 0. For every A and B , print their product. (5 marks) [CSE 101 | L-1/T-1 | 2015–16]

Q18. Write a program that takes a float x and integer n as input and calculates the value of the series:

$$x - \frac{x^2}{2!} + \frac{x^3}{3!} - \frac{x^4}{4!} + \dots \pm \frac{x^n}{n!}$$

You are not allowed to use any library functions except I/O-related ones. (12 marks)

[CSE 101 | L-1/T-1 | 2015–16]

Q19. Suppose an array has n positive integers ($n \geq 2$). Show a code segment to find the GCD of all numbers in the array. (8 marks) [CSE 101 | L-1/T-1 | 2015–16]

Q20. July, 20, 1973 is a FRIDAY. There is a formula to find day of the week for a particular date. Zeller's congruence is an algorithm developed by Christian Zeller to calculate the day of the week. The formula is as follows:

$$h = \left(q + \left\lfloor \frac{26(m+1)}{10} \right\rfloor + y + \left\lfloor \frac{y}{4} \right\rfloor + 6 \times \left\lfloor \frac{y}{100} \right\rfloor + \left\lfloor \frac{y}{400} \right\rfloor \right) \% 7$$

Where, h is the day of week (0: Saturday, 1: Sunday, 2: Monday, 3: Tuesday, 4: Wednesday, 5: Thursday, 6: Friday).

q is the day of the month

m is the month (3: March, 4: April, 12: December). January and February are counted as months 13 and 14 of the previous year.

y is the year.

$\lfloor \cdot \rfloor$ is called floor operator. When it is applied to a fractional number x it returns the largest integer not exceeding x . For example, $\lfloor 2.3 \rfloor = 2$, $\lfloor 2.7 \rfloor = 2$ and $\lfloor 2.0 \rfloor = 2$

NOTE: In this algorithm January and February are counted as months 13 and 14 of the previous year. E.g. if it is February 2, 2010, the algorithm counts the date as the second day of the fourteenth month of 2009 (02/14/2009 in DD/MM/YYYY format). Write down a program that takes year, month and day of the month as input and prints the day of the week. (15 marks) [CSE 105 | L-1/T-2 | 2014–15]

Q21. Most positive integers can be expressed as a sum of two prime numbers (e.g., $9 = 2 + 7$). Write a C program that takes an integer n as input and checks whether n can be expressed as a sum of two primes or not. (11 marks) [CSE 105 | L-1/T-2 | 2014–15]

Q22. An analog clock has only two hands - hour and minute. The input for the program will be a set of 24 hour format time. For the given input the program should output all the input time and the angle created for that particular time in ascending order.

The input for the program will be given in the following order. The first line will contain an integer N where $1 \leq N \leq 100$. The next N lines will contain two integers hh ($0 \leq hh \leq 23$) and mm ($0 \leq mm \leq 59$) separated by a blank space indicating hour and minute of a particular time in 24 hours format. For the input set the program should provide N lines as output. Each line should contain three integers separated by single blank space. The first two numbers are the hour and minute of the input set and the last number (rounding to integer) is the angle between the hour and minute hands. These three numbers should appear in the ascending order of the angle between the hour and minute hands. If there are times with the same angle then the earlier time should appear first (00:00 is the earliest time and 23:59 is the latest time).

Sample input and output for the program may be:

Sample input	Sample output
4	0 0 0
3 0	23 59 6
9 0	3 0 90
23 59	9 0 90
0 0	

In the program sections there is one user defined structure named **Time**. This structure has three integer variables **hh**, **mm**, and **angle**; and one long integer value **sortingKey**. **hh** and **mm** are used to store the hour value and minute value of the time. Variable **angle** holds the value of the angle $\leq 180^\circ$ (acute and obtuse angle only) created by the hour and minute hands of the clock. Variable **sortingKey** holds a value that is used to sort the list of time according to the problem statement.

The program contains a function with prototype **void SortTime(int i)**. The description of the function is as follows—

void SortTime(int i). This function basically combines three different numbers (**angle**, **hh** and **mm**) to get a value and set that value to the **sortingKey** variable of a particular element having index *i*. For example: if the *i*-th element of the array **t[]** has **hh** = **16**, **mm** = **55** and **angle** = **177** the function will combine the numbers **177**, **16** (from **hh**) and **55** (from **mm**) to get the value **1771655** and then sets this value to the variable **sortingkey** of the *i*-th element. When the total list is sorted in the ascending order of this **sortingKey** the list is eventually sorted in the ascending order of **angle** and then by time.

```

1 void main()
2 {
3     long n, i, j, hour, totalMin;
4     float hAngle, mAngle, angle;
5     scanf("%ld", &n);
6     for(i = 0; i < n; i++){
7         scanf("%ld %ld", &t[i].hh, &t[i].mm);
8         hour = t[i].hh;
9         if(      A      ){
10             hour = t[i].hh - 12;
11         }
12         totalMin =      B      ;
13         hAngle = round(totalMin * 0.5);
14         mAngle = t[i].mm * 6;
15         if(hAngle > mAngle){
16             t[i].angle = hAngle - mAngle;
17         }
18         else{
19             t[i].angle = mAngle - hAngle;
20         }
21         if(t[i].angle > 180){
22             C
23         }
24         SortTime(i);
25     }
26     Sort((void *) &t, n, sizeof(Time));
27     for(i = 0; i < n; i++){
28         printf("%d %d %ld\n", t[i].hh, t[i].mm, t[i].angle);
29     }
30 }
```

(a) Fill up the blank A, B and C. (12)

(b) Write down the structure defined in the problem description with 5 bits for hour, 6 bit for minute, 8 bit for angle and required minimum number of bits for **sortingKey**. Put **sortingKey** in a new word. Use **typedef** to redefine it to a name of **Time**. (7+2=9)

What is the value of **sizeof(Time)**?

(c) Write down the codes for the **Sort()** function which is referenced in the main function. (14)

[CSE 105 | L-1/T-2 | 2013–14]

Q23. Write a program to determine the summation of the series:

$$\log_2 2 - \log_2 4 + \log_2 8 - \dots + \log_2 n$$

Assume $n = 2^m$ for some integer $m > 0$; m is the input. You are not allowed to use any library function except I/O-related ones.

(10 marks)

[CSE 105 | L-1/T-2 | 2013–14]

Q24. Twin primes are pairs of prime numbers that differ by 2 (e.g., (3,5), (5,7), (11,13)). Write a program to find all twin primes less than a positive integer $n > 1$ entered by the user, and display them one pair per line. (12 marks) [CSE 105 | L-1/T-2 | 2013–14]

Q25. Write a complete C program that reads an integer n and computes the term a_n where $a_1 = 1$, $a_2 = 2$, $a_3 = 3$, and for $k > 3$: $a_k = a_{k-2} + 2 \times a_{k-3}$. Assume $n > 0$. You may use a loop, but you are not allowed to use arrays, nested loops, or recursion.

Sample: Enter n: 5 $\rightarrow$ a_5 is 7 (15 marks)

[CSE 105 | L-1/T-2 | 2012–13]

Q26. Rewrite the following statements so that a **do-while** loop is used instead of the **while** loop (results must be identical):

```

1  int sum = 0, num;
2  scanf("%d", &num);
3  while (num >= 0) {
4      sum += num;
5      scanf("%d", &num);
6  }
    
```

(7 marks)

[CSE 105 | L-1/T-2 | 2012–13]

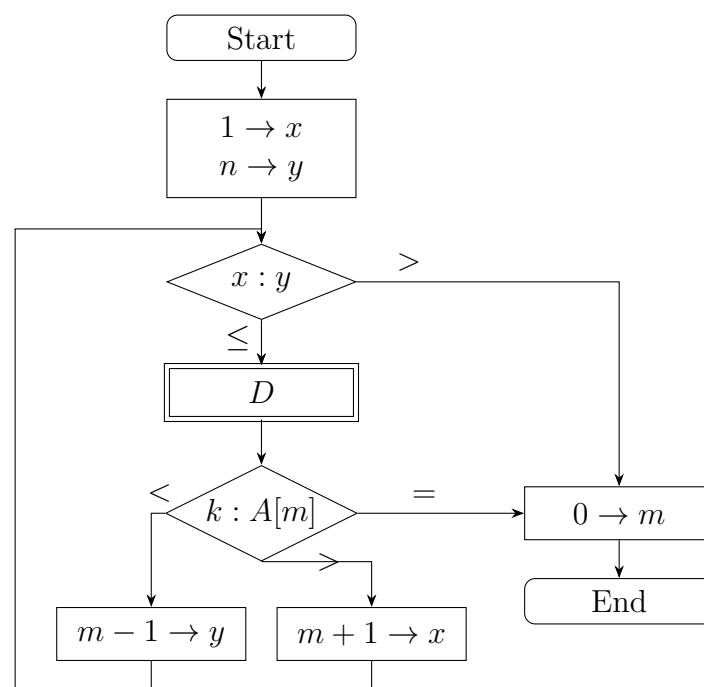
Q27. Write an interactive C program that will convert a date entered in the form **mm-dd-yy** (example: 4-12-13) into an integer indicating the number of days beyond January 1, 2000. Use the following relationships (valid for $yy \leq 99$):

- (a) $\text{day} = (\text{int})(30.42 * (\text{mm} - 1)) + \text{dd}$
- (b) If $\text{mm} == 2$, increase **day** by 1.
- (c) If $\text{mm} > 2$ and $\text{mm} < 8$, decrease **day** by 1.
- (d) If $\text{yy} \% 4 == 0$ and $\text{mm} > 2$ (leap year), increase **day** by 1.
- (e) Increase **day** by 1461 for each full 4-year cycle beyond 1-1-2000.
- (f) Increase **day** by 365 for each additional full year beyond the last full 4-year cycle, then add 1 for the most recent leap year.

(12 marks)

[CSE 105 | L-1/T-2 | 2011–12]

Q28. The flowchart below shows a binary search algorithm to find the index m of the array element $A(m)$, such that the equation " $A(m) = k$ " holds, from the array elements $A(1), A(2), \dots, A(n)$ already sorted in ascending order. In case of " $m = 0$ " at the end, there is no element such that the equation " $A(m) = k$ " holds. Which statement is to be inserted in the process box D in the flowchart? (10)



Write an iterative C function for implementing the flowchart stated in the figure.

(10 marks)

[CSE 105 | L-1/T-2 | 2011–12]

Q29. An Armstrong number of three digits is an integer such that the sum of the cubes of its digits equals the number itself (e.g., $153 = 1^3 + 5^3 + 3^3$). Write a C program to print all Armstrong numbers between 100 and 999. (10 marks) [CSE 105 | L-1/T-2 | 2010–11]

Q30. Write a program to find whether N is a super-prime or not. A number is super-prime if it and all its left-truncations are prime (e.g., 7331 is super-prime because 7331, 733, 73, and 7 are all prime; 4550 is not). (15 marks) [CSE 105 | L-1/T-2 | 2010–11]

Q31. Write a program that can evaluate a mathematical expression of any length entered by a user, with $+$, $-$, $*$, and $/$ operators and double-type operands in infix notation. For example: $4.0*6.5+32.0/4.0-10.45+5.5*7.25$. (10 marks) [CSE 101 | L-1/T-2 | 2009–10]

Q32. Write a C program that will generate a table of values for the equation

$$y = 2e^{-0.1t} \sin(0.5t)$$

where t varies from 0 to 60. Allow the size of the t -increment to be entered as an input parameter. Use a macro to evaluate the formula. (10 marks) [CSE 101 | L-1/T-2 | 2009–10]

Q33. Write a C program that determines the value of x which causes the function $y = x \cos(x)$ to be maximized within a specified interval. (8 marks) [CSE 101 | L-1/T-2 | 2008–09]

Q34. Write a simple program that will produce the following multiplication table using a loop.

1	2	3	4	5	6
2	4	6	8	10	12
3	6	9	12	15	18
4	8	12	16	20	24
5	10	15	20	25	30
6	12	18	24	30	36

(10 marks)

[CSE 105 | L-1/T-2 | 2005–06]

Q35. Write a program that takes an integer n as input. If the number is even, divide it by 2; otherwise multiply it by 3 and add 1. Continue until $n = 1$. Output the number of times the operations were executed. (For example, if $n = 3$ the answer is 7.) (7 marks)
[CSE 101N | L-1/T-1 | 2003–04]

Q36. For the following code sample, what is the final value of i ?

```
1 i = 0;
2 do {
3     i = i + 1;
4     if (i < 10) continue;
5     i = i - 1;
6 } while (++i < 6);
```

(4 marks)

[CSE 101N | L-1/T-1 | 2003–04]

Q37. When using the `continue` statement, the `for` loop behaves slightly differently from the corresponding `while` or `do-while` loop. Explain how. (4 marks)
[CSE 101N | L-1/T-1 | 2003–04]

Q38. Evaluate the cosine series:

$$\cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots$$

Assume x ($0 < x < \pi$) is given in radians. Compute the value to some arbitrary precision using a loop. (6 marks) [CSE 101N | L-1/T-1 | 2003–04]

Nested Control, break & continue

Q1. Assuming $N = 6$, analyse the following nested loop and find the corresponding output:

```
1 for (i = 1; i <= N + 1; i++) {
2     for (j = 1; j <= N; j++) {
3         printf("%c", 'A' + ((i + j - 2) % N));
4     }
5     printf("\n");
6 }
```

(10 marks)

[CSE 101 | L-1/T-1 | 2023–24]

Q2. What is the output of the following code? Assume i and n are integers.

```
1 for (n = 1; n <= 5; n++)
2     for (i = 1; i < n; i++)
3         printf("%d%c", i, (i == n-1) ? '\n' : ',');
```

(5 marks)

[CSE 101 | L-1/T-1 | 2018–19]

Q3. Using nested loops, print an $N \times N$ grid where: (i) odd rows have N plus signs and even rows have N minus signs; (ii) odd columns have N plus signs and even columns have N minus signs. N is input. (Show only the loop nesting, not the entire program.)

Example for 2(i): (suppose N=5)	Example 2(ii): (suppose N=5)
+ + + + +	+ - - - +
+ + + + +	- + - + -
+ + + + +	- - + - -
+ + - - -	- + - + -
+ + + + +	+ - - - +

(10 marks)

[CSE 101 | L-1/T-1 | 2015–16]

Q4. Write a program that takes a positive integer n as input and displays a pyramid pattern.

```
1 //For n=5,
2     +
3   + ^ +
4 + ^ ^ ^ +
5 + ^ ^ ^ ^ +
6 + + + + + + + +
```

(12 marks)

[CSE 101 | L-1/T-1 | 2015–16]

Q5. Rewrite the following code using `do-while` loops only:

```
1 for (i = 1; i <= 5; i++) {
2     for (j = 1; j <= i; j++)
3         printf("%i", i);
4     printf("\n");
5 }
```

(5 marks)

[CSE 105 | L-1/T-2 | 2010–11]

goto Statement

Q1. Why is it not encouraged to use the `goto` statement? Rewrite the following code segment without using `goto`:

```
1 int a, b, c;
2 scanf("%d %d", &a, &b);
3 if (a < 0) goto label1;
4 if (b > 0) goto label2;
5 c = a;
6 goto label1;
7 label2: c = b;
8 label1:
```

(10 marks)

[CSE 105 | L-1/T-2 | 2007–08]

Q2. Why is the use of the `goto` statement discouraged in structured programming? State a case in which `goto` is more useful than other control statements. (5 marks)

[CSE 101N | L-1/T-1 | 2003–04]

BUET · CSE 101 · Structured Programming

Part II

Functions & Arrays

Functions & Arrays

S3 — Functions

Function Basics & Parameter Passing

Q1. What is a function prototype? Explain with an example. Why do we need it? (5 marks) [CSE 101 | L-1/T-1 | 2024–25, 2023–24]

Q2. Write down the output of the following program:

```
1 #include <stdio.h>
2
3 void swap(int a, int b) {
4     int *addressA = &a;
5     int *addressB = &b;
6     int *temp;
7     temp = addressA;
8     addressA = addressB;
9     addressB = temp;
10 }
11
12 int main() {
13     int a = 10;
14     int b = 30;
15     swap(a, b);
16     printf("A=%d, B=%d", a, b);
17     return 0;
18 }
```

(7 marks)

[CSE 101 | L-1/T-1 | 2022–23]

Q3. What is the difference between pass by value and pass by reference? (5 marks) [CSE 101 | L-1/T-1 | 2022–23]

Q4. Write down the output produced by the printf statements of the following main function:

```
1 void swap(int x, int y, int *count) {
2     static int i = 0;
3     int t = x; x = y; y = t;
4     i++;
5     *count = i;
6 }
7 int main() {
8     int a = 4, b = 5, count;
9     swap(a, b, &count);
10    printf("Swapping count %i %i %i", a, b, count);
11    count = 3;
12    swap(a, b, &count);
13    printf("Swapping count %i %i %i", a, b, count);
14 }
```

(3 marks)

[CSE 101 | L-1/T-1 | 2021–22]

Q5. Write the following two functions for a degree- n polynomial $p(x) = a_0 + a_1x + \cdots + a_nx^n$:

(a) double eval(double a[], double x, int n) — returns $p(x)$ evaluated at x .

(b) void add(double f[], double g[], double h[], int n) — stores the sum of polynomials g and h (both of degree $\leq n$) in f .

(12 marks)

[CSE 101 | L-1/T-1 | 2018–19]

Q6. Write a utility function kSmooth(x, k) that returns 1 if x is k -smooth (has no prime factors greater than k), and 0 otherwise.

k	k -smooth numbers
2	1, 2, 4, 8, 16, 32, 64, 128, 256, 512, ...
3	1, 2, 3, 4, 6, 8, 9, 12, 16, 18, 24, ...
4	1, 2, 3, 4, 5, 6, 8, 9, 10, 12, 15, 16, ...
5	1, 2, 3, 4, 5, 6, 8, 9, 10, 12, 15, 16, ...

(13 marks)

[CSE 101 | L-1/T-1 | 2015–16]

Q7. Write a function int isPowerOfFive(int n) that returns 1 if n is a power of 5 and 0 otherwise. You are not allowed to use any library functions.

Function call	Return value
isPowerOfFive(5)	1
isPowerOfFive(10)	0
isPowerOfFive(25)	1
isPowerOfFive(33)	0

(11 marks)

[CSE 101 | L-1/T-1 | 2015–16]

Q8. Write a function `double horner(double a[], int n, double x)` that uses Horner's rule to evaluate the polynomial $p(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$. The array `a` has $n+1$ elements (a_0 to a_n); rewrite the evaluation as $a_0 + x(a_1 + x(a_2 + \dots x(a_{n-1} + x \cdot a_n) \dots))$.

(12 marks)

[CSE 105 | L-1/T-2 | 2014–15]

Q9. Write a function `int checkFibonacci(int n)` that returns 1 if `n` is a Fibonacci number and 0 otherwise. (`checkFibonacci(8) → 1`; `checkFibonacci(21) → 1`; `checkFibonacci(7) → 0`.)

(10 marks)

[CSE 105 | L-1/T-2 | 2014–15]

Q10. Given two integers n and x as parameters where $n > x > 1$, write a C function `int powerOfX(int n, int x)` that returns y if $n = x^y$ for some $y \geq 0$, and -1 otherwise. For example, `powerOfX(125, 5)` returns 3 since $125 = 5^3$.

(10 marks)

[CSE 105 | L-1/T-2 | 2013–14]

Q11. A Fermat number has the form $F_t = 2^{(2^t)} + 1$ (e.g., 3, 5, 17, 257, ...). Write a program that takes an integer y as input and determines whether y is a Fermat number, using the `powerOfX(n, x)` function from the previous question.

(10 marks) [CSE 105 | L-1/T-2 | 2013–14]

Q12. Write a program to calculate $\sqrt{x}$ where x is a positive double value. You cannot use any library function.

(10 marks)

[CSE 105 | L-1/T-2 | 2013–14]

Q13. Write down a C program to calculate $\sqrt{|x|}$ where x is a double value. You cannot use any library function.

(10 marks)

[CSE 105 | L-1/T-2 | 2012–13]

Q14. Describe the output of the following program:

```

1  #include <stdio.h>
2  int a = 0, b = 1;
3  int funct1(int a);
4  int funct2(int b);
5  main() {
6      int count;
7      for (count = 1; count <= 5; ++count) {
8          b += funct1(a + 1) + 1;
9          printf("%d ", b);
10     }
11 }
12 funct1(int a) {
13     b = funct2(a + 1) + 1;
14     return (b);
15 }
16 funct2(int a) {
17     return (b + a);
18 }
```

(15 marks)

[CSE 105 | L-1/T-2 | 2011–12]

Q15. What is a function prototype? Explain with an example. Write a C function named `swap` which inter-exchanges the values of `x` and `y` without declaring any variable within its body.

(12 marks)

[CSE 105 | L-1/T-2 | 2011–12]

Q16. Describe the output of the following program:

```

1  #include <stdio.h>
2  int a = 0, b = 1;
3  int funct1(int a);
4  int funct2(int b);
5  main() {
6      for (int count = 1; count <= 5; ++count) {
7          b += funct1(a + 1) + 1;
8          printf("%d ", b);
9      }
10 }
11 int funct1(int a) {
12     static int c = 0;
13     b = funct2(a + c++) + 1;
14     return (b);
15 }
```

```

16 int funct2(int a) {
17     if (a == 1) return (b + a);
18     return (b + funct2(a - 1));
19 }

```

(15 marks)

[CSE 101 | L-1/T-2 | 2008–09]

Q17. Write a function that can be used to sort any data type using the Quick Sort algorithm. Write also any necessary helper functions. Specify a comparison function for sorting structured data containing `name` (string) and `age` (int): sort in descending order of age; break ties by ascending order of name. (20 marks)

[CSE 105 | L-1/T-2 | 2007–08]

Q18. What is the difference between “call by value” and “call by reference”? Explain with an example. (8 marks)

[CSE 105 | L-1/T-2 | 2005–06]

Q19. What does the `return` statement in the `main()` function indicate? (5 marks)

[CSE 105 | L-1/T-2 | 2005–06]

Q20. “In C, a call by reference is in fact one kind of call by value” — explain this statement. (5 marks)

[CSE 101N | L-1/T-1 | 2003–04]

Recursion

Q1. Write a recursive function `int strlen(char s[])` that recursively determines and returns the length of a string. For example, `strlen("DON'T UNDERESTIMATE THE POWER OF A COMMON MAN")` should return 45. (10 marks)

[CSE 101 | L-1/T-1 | 2023–24]

Q2. Write a recursive function in C to calculate the sum of the even digits of a positive integer. For example: input 123 → output 2; input 456 → output 10; input 135 → output 0. (15 marks)

[CSE 101 | L-1/T-1 | 2022–23]

Q3. Consider the following recursive function. Draw the recursive tree for the call `fib(6)` and write the output of the program:

```

1 long fib(int num) {
2     if (num <= 3) return 1;
3     long val1 = fib(num-1);
4     printf("%ld ", val1);
5     long val2 = fib(num-2);
6     printf("%ld ", val2);
7     long val3 = fib(num-3);
8     printf("%ld ", val3);
9     return val1 + val2 + val3;
10 }
11 int main() {
12     long ret = fib(6);
13     printf("\n%ld", ret);
14 }

```

(10 marks)

[CSE 101 | L-1/T-1 | 2021–22]

Q4. Write a recursive function `int strcmp(char s[], char t[], int i)` that compares two strings and returns 0 if equal, `-1` if `s` lexicographically precedes `t`, and 1 otherwise. An initial call `strcmp(s, t, 0)` starts comparison from the beginning. (15 marks)

[CSE 101 | L-1/T-1 | 2019–20]

Q5. Write a recursive function `int largest(int x[], int n)` that returns the largest element in an array of `n` integers. Do not use loops, static, or global variables. Also write a simple `main` function demonstrating the initial call. (15 marks)

[CSE 101 | L-1/T-1 | 2018–19]

Q6. Write a recursive function `void printBinary(unsigned int x)` that prints the binary representation of `x` without leading zeros (except for 0 itself). Do not use loops, static, or global variables. No pre-processing in `main()`. (15 marks)

[CSE 101 | L-1/T-1 | 2018–19]

Q7. Consider the following `enigma` function:

```

1 int enigma(int n) {
2     if (n == 1) return 0;
3     else return 1 + enigma(n / 2);
4 }

```

What will be the output if `enigma(4096)` is called from `main`? In one sentence, state what the function does. Assume $n > 0$. (7 marks)

[CSE 101 | L-1/T-1 | 2015–16]

Q8. Write a C function `decToBin(int num)` that returns the binary equivalent of a given integer using recursion. (8 marks)

[CSE 101 | L-1/T-1 | 2015–16]

| L-1/T-1 | 2015–16]

Q9. Write a recursive C function for binary search within a sorted list that returns the index of the element if found, or -1 otherwise.

```
1 int binSearch(int* list, int low, int high, int value);
```

where list is the name of the sorted array and value is the element to be searched. Variables low and high represent the searching range. **(10 marks)** [CSE 101 | L-1/T-1 | 2015–16]

Q10. Consider the following `enigma` function:

```
1 void enigma(int n) {
2     if (n != 0) enigma(n / 2);
3     printf("%d", n % 2);
4 }
```

What will be the output if `enigma(13)` is called from `main`? In one sentence, state what the function does. **(7 marks)** [CSE 105 | L-1/T-2 | 2014–15]

Q11. Calculate the value of $f(4)$ for the following recursive function:

$$f(0) = 1, \quad f(n) = n \times f(n-1) + n$$

(6 marks)

[CSE 105 | L-1/T-2 | 2014–15]

Q12. Write a recursive C function `int count(char s[], char c)` that returns the number of occurrences of character `c` in string `s`. For example, `count("Toronto", 't') → 1`; `count("Toronto", 'o') → 3`. You must use recursion; no loops. **(12 marks)** [CSE 105 | L-1/T-2 | 2014–15]

Q13. The Towers of Hanoi is played with three poles and n disks of different sizes, initially stacked on the leftmost pole in decreasing size. The object is to transfer all disks to the rightmost pole, moving only one disk at a time and never placing a larger disk on a smaller one. Write a recursive C function that solves this problem and describe the output for $n = 4$. Also calculate the number of moves required to move n disks. **(15 marks)** [CSE 105 | L-1/T-2 | 2012–13]

Q14. A table contains n unique data elements sorted in ascending order. The search algorithm splits the table into blocks of m elements and performs a linear search on the last element of each block to find the target block. If the selected block has fewer than m elements, a linear search is done on that block; otherwise the block is split into sub-blocks of m and the process repeats recursively. Write a recursive C function implementing this searching algorithm for a list of integers. **(10 marks)** [CSE 105 | L-1/T-2 | 2011–12]

Q15. Write a recursive C function to solve the Legendre Polynomials:

$$P_n = \frac{2n-1}{n} x P_{n-1} - \frac{n-1}{n} P_{n-2}$$

where $P_0 = 1$, $P_1 = x$, and x is any floating-point number in $[-1, 1]$. Describe the output of the function for $n = 5$ and $x = 0.5$. **(13 marks)** [CSE 105 | L-1/T-2 | 2011–12]

Q16. The Legendre polynomials can be calculated by the formulas:

$$P_0 = 1, \quad P_1 = x, \quad P_n = \frac{2n-1}{n} x P_{n-1} - \frac{n-1}{n} P_{n-2} \quad (n = 2, 3, 4, \dots)$$

where x is any floating-point number in $[-1, 1]$. Write a C function `LP(n, x)` which returns the n -th Legendre polynomial value. **(10 marks)**

Explain the output generated by the recursive function `LP(n, x)` described above for $n = 6$. **(15 marks)** [CSE 101 | L-1/T-2 | 2008–09]

Q17. What is a recursive function? Describe its positive and negative aspects. **(10 marks)** [CSE 105 | L-1/T-2 | 2007–08]

Q18. Let `array` be an external integer variable declared and initialized as follows. A *cluster* of 1s in the array is defined recursively: search each position with value 1, and for each such position, search its eight neighbours for 1s, repeating recursively. Write a function that returns the number of clusters in the array. For example, the following array contains two clusters:

```
1 int array[5][5] = {
2     {1, 0, 0, 0, 1},
3     {0, 1, 0, 1, 0},
4     {0, 0, 1, 0, 0},
5     {0, 1, 0, 1, 0},
6     {1, 0, 0, 0, 1}
7 };
```

(15 marks)

[CSE 105 | L-1/T-2 | 2007–08]

Q19. Write a recursive function prototyped as `void itob(char s[], int i)` which converts the integer `i` to its binary equivalent string in `s`. **(10 marks)** [CSE 105 | L-1/T-2 | 2007–08]

Q20. Consider the following program code:

```
1 int func(int a, int b) {
2     if (b == 1) return a;
3     else return a + func(a, b - 1);
4 }
```

- What does the above program do?
- What will be the output if `func(4, 4)` is called from `main()`?
- What will happen if `b` is less than 0?
- If the program does not give correct output for `b < 0`, fix the problem and rewrite the function.

(18 marks)

[CSE 105 | L-1/T-2 | 2005–06]

Q21. “Any problem that can be solved recursively can also be solved iteratively.” Explain this statement with an example. Why and when will you prefer recursion to iteration? **(12 marks)** [CSE 105 | L-1/T-2 | 2005–06]

Q22. You are given an already-built linked list of five nodes:

```
1 struct node {
2     int x;
3     struct node *pNext;
4 };
```

The values in consecutive nodes starting at head `h` are 1, 2, 3, 4, and 5. Write a recursive function `void PrintList(struct node *ptr)` that prints the list in reverse order (i.e. outputs 5, 4, 3, 2, 1). **(8 marks)** [CSE 101N | L-1/T-1 | 2003–04]

Q23. The sequence $T_n = \sum_{i=1}^n i$ generates the triangular numbers. The first four are 1, 3, 6, and 10. Write a recursive function `int Triangle(int n)` that takes a positive integer `n` and returns T_n . **(7 marks)** [CSE 101N | L-1/T-1 | 2003–04]

S4 — Arrays & Strings

One-Dimensional Arrays: Searching & Sorting

Q1. Write down a program that will take a set of integers as input in an array and will print only duplicate elements in the array. A duplicate element must be printed once. Also, you are not allowed to **change the content** of the array or use any **additional array**. The number of elements in the set, denoted by N , will also be input to your program.

Sample Input/output

N: 8

Enter array elements: 3 5 6 7 3 7 9 3

Duplicates: 3 7

(17 marks)

[CSE 101 | L-1/T-1 | 2024–25]

Q2. Write a C program to input elements in a character array and find frequency of each character in the array. Assume that only capital letters will be input to your program. The number of characters will also be input to your program. Please also note the following couple of constraints:

- Your program should have only one function called `main()` and no other functions.
- You cannot use any nested loop to solve this problem but you can use more than one array if needed.

Sample Input/output

Input size of array : 10

Input array elements: A B C A B D A Z X Z

Output:

Frequency of A = 3

Frequency of B = 2

Frequency of C = 1

Frequency of D = 1

Frequency of X = 1

Frequency of Z = 2 **(18 marks)**

[CSE 101 | L-1/T-1 | 2024–25]

Q3. Write a C program that takes an array of integers as input and prints the following properties:

- (a) **U/A/D** — whether the array is Unsorted (U), sorted in Ascending (A) order, or Descending (D) order.
- (b) **k** — the largest difference between any two array elements.
- (c) **f** — the sum of the unique (non-duplicate) elements of the array.

The output should be one character (U/A/D) followed by two integers (k and f). **(20 marks)** [CSE 101 | L-1/T-1 | 2023–24]

Q4. Research is a fascinating thing. The highest number of citations in a published research article doesn't really tell the full story about a researcher. In academia, we use a metric called h-index that measures both the productivity and citation impact of the publications of a researcher. It is defined as follows - the largest number h such that h articles have at least h citations each. For example, if an author has five publications, with, 9, 5, 4, 2, and 1 citations (ordered from greatest to least), then the author's h-index is 3, because the author has three publications with 3 or more citations. Write down a function `int calculateHindex(int C[], int N)` which takes an array C containing the list of citations of the papers of an author, the size of the array N and calculates the h-index of the author. Assume that the array elements will always be ordered from greatest to least. **(15 marks)** [CSE 101 | L-1/T-1 | 2023–24]

Q5. Write a C program that takes an integer N as input, then reads N numbers into an array, and prints the duplicate numbers (numbers that appear more than once).

Sample Input: 5 / 1 2 2 3 1 *Sample Output:* The duplicates are: 1 2 **(12 marks)** [CSE 101 | L-1/T-1 | 2022–23]

Q6. Write a program that prints all elements present in array A but **not** in array B (set difference $A \setminus B$). The sizes and elements of both arrays are taken as input; all elements within each array are unique. For example, if $A = \{15, 7, 9\}$ and $B = \{1, 2, 3, 15\}$, the output is 7, 9. **(13 marks)** [CSE 101 | L-1/T-1 | 2022–23]

Q7. Describe different ways to declare an integer array of size 100. **(3 marks)** [CSE 101 | L-1/T-1 | 2021–22]

Q8. Write a function that takes four parameters: (i) an array, (ii) an element to insert, (iii) the index at which to insert it, and (iv) the current size of the array. The function inserts the element at the specified position. **(8 marks)** [CSE 101 | L-1/T-1 | 2021–22]

Q9. Write a C function `int deleteSorted(int x)` that deletes element x from a sorted integer array A of N elements (both declared globally). Returns the index from which the element was deleted, or -1 if not found. Assume no duplicate elements. **(8 marks)** [CSE 101 | L-1/T-1 | 2020–21]

Q10. Write a complete C program to sort the following array of structs alphabetically by name. Break ties by placing the student with the lower ID first:

```
1 struct student { int id; char name[50]; };
2 struct student students[50];
```

(15 marks) [CSE 101 | L-1/T-1 | 2020–21]

Q11. Write a program that prints all elements present in array A but not in array B (i.e., $A \setminus B$). Both arrays contain unique elements; their sizes and values are input. For example, $A = \{15, 7, 9\}$, $B = \{1, 2, 3, 15\} \rightarrow$ output 7, 9. **(15 marks)** [CSE 101 | L-1/T-1 | 2019–20]

Q12. Write a function `void remove(int A[], int N, int V)` that searches if V exists in array A of N integers and removes the **last** occurrence of V , shifting each following element left and placing a zero at the end. **(12 marks)** [CSE 101 | L-1/T-1 | 2015–16]

Q13. Given two unsorted arrays A (size m) and B (size n), implement:

- (a) `void union(int A[], int B[], int C[], int m, int n)` — computes $C = A \cup B$.
- (b) `void intersect(int A[], int B[], int D[], int m, int n)` — computes $D = A \cap B$.

(18 marks) [CSE 101 | L-1/T-1 | 2015–16]

Q14. Write a function `void remove(int A[], int N, int V)` that searches if V exists in array A of N integers and removes the first occurrence of V , shifting all subsequent elements left and placing a zero at the end. **(12 marks)** [CSE 105 | L-1/T-2 | 2014–15]

Q15. Using the `qsort()` library function, you need to sort a list of strings lexicographically. Your input contains several lines. Each line contains a string (may contain spaces) of length no more than 32 characters. Process the input until the end of file is reached. The total number of strings will not exceed 150. After sorting, print out the strings according to the sort order. Recall that the `qsort()` library function has the following prototype:

```
1 void qsort(
2     void *Base,
3     unsigned int NumOfElements,
4     unsigned int SizeOfElements,
5     int (*PtFuncCompare)(const void *, const void *))
6 ;
```

(10 marks) [CSE 105 | L-1/T-2 | 2014–15]

- Q16.** Complete the definition of the C function `int maxCount(int list[], int length)` that returns the number of times the global maximum appears in the array. For example, in `{3,9,7,5,9,8,2,4,9}`, the global maximum is 9 and it appears 3 times, so the function returns 3. **(12 marks)** [CSE 105 | L-1/T-2 | 2013–14]
- Q17.** Write a function `int nonrepeated(int a[], int n)` that returns the number of non-repeated elements in a sorted array `a` of length `n` (elements in ascending order, at least one element). **(13 marks)** [CSE 105 | L-1/T-2 | 2013–14]
- Q18.** Write a C function `int countDistinct(int list[], int listLength)` where `list` is an integer array sorted in ascending (non-decreasing) order and `listLength` is its size. The function counts and returns the number of distinct elements. For example, `{1,1,3,3,3,3,6,8,8,9,9,9,9}` $\rightarrow$ 5. **(15 marks)** [CSE 105 | L-1/T-2 | 2012–13]
- Q19.** Write a C function to delete an integer element from a sorted array. **(10 marks)** [CSE 105 | L-1/T-2 | 2011–12]
- Q20.** Write a program to enter n elements in an array and find the second smallest number from the array. There must be no nested loops in your program. **(10 marks)** [CSE 105 | L-1/T-2 | 2010–11]
- Q21.** Write a print function to print the elements of a triangular array (see dynamic memory allocation questions) from the top to the bottom. **(7 marks)** [CSE 101 | L-1/T-2 | 2009–10]
- Q22.** In merge sort, sorting a sequence S with n elements uses three divide-and-conquer steps: (i) if S has zero or one element, return immediately; (ii) divide S into two sequences S_1 and S_2 each of about $n/2$ elements; (iii) recursively sort S_1 and S_2 , then merge them. Implement this merge sort algorithm for a one-dimensional array of n elements. **(10 marks)** [CSE 101 | L-1/T-2 | 2009–10]
- Q23.** Consider a 2-dimensional array of integers which are row-wise and column-wise unsorted. Write a C function that sorts the integers such that the array becomes row-wise sorted. **(10 marks)** [CSE 101 | L-1/T-2 | 2008–09]
- Q24.** Write a function that takes an array of integers and its size as parameters and returns the element that appears the maximum number of times. For example, for `a[] = {1,5,4,4,2,6,4}` it returns 4 (appears 3 times). For multiple such elements, return the first one. **(8 marks)** [CSE 101N | L-1/T-1 | 2003–04]

Character & String Operations

- Q1.** "Array of characters and string are same" - Justify. Explain different ways to represent string with examples. **(7 marks)** [CSE 101 | L-1/T-1 | 2024–25]
- Q2.** Take two strings as input. Next, from the first string, delete the characters that are in the second string. Output the remainder of the first string. **(8 marks)** [CSE 101 | L-1/T-1 | 2024–25]
- Q3.** You are to implement `strchrs` function, which can search for more than one characters in a string and returns the location of the characters. In this case, all the search keyword characters are to be given within another string as parameter. Every time `strchrs` is called, it returns the index of the string containing the location of one of the search characters in the string. At first the function will be called by the string as parameter and it will return the index of the first location. To get the subsequent location NULL is passed as parameter to `strchrs`. If a new string is given instead of NULL, then finding subsequent locations takes place on the new string. If there is no other characters in the string it will return -1. **(12 marks)** [CSE 101 | L-1/T-1 | 2024–25]
- Q4.** Write functions to implement the following:
- (i) copying a string to another string
 - (ii) Comparing two strings
- (4 × 2 = 8 marks)** [CSE 101 | L-1/T-1 | 2024–25]
- Q5.** What is the necessity of the null character (`'\0'`) in defining a string in C? Explain with necessary examples. **(6 marks)** [CSE 101 | L-1/T-1 | 2023–24]
- Q6.** Implement the `strtok` function that can search for more than one separator character in a string, where all separator characters are provided as a second string parameter. The first parameter is the string to be tokenised. Each call returns the address of the next token. On the first call, pass the source string; on subsequent calls, pass NULL to continue tokenising. If a new non-NULL string is passed, tokenisation restarts on that string. **(12 marks)** [CSE 101 | L-1/T-1 | 2023–24]
- Q7.** You are given the hexadecimal string "A7B8". First convert it to a decimal number, then convert that number to an octal string and output it. For "A7B8" the expected output is "123670". **(9 marks)** [CSE 101 | L-1/T-1 | 2023–24]
- Q8.** Write a C function that converts an input integer into its equivalent string representation. For example, input 523 should produce the string "523". The scope of the returned string must be accessible from `main`. **(15 marks)** [CSE 101 | L-1/T-1 | 2022–23]
- Q9.** Complete the following C code to reverse a string **without** using any library function from `<string.h>`. Write your code with proper comments for the missing part only.

```

1  #include <stdio.h>
2  int main() {
3      char str[100], reversedStr[100];
4      gets(str);
5      // add your code here
6      // start of your code
7
8      // end of your code
9      puts(reversedStr);
10     return 0;
11 }
    
```

(12 marks)

[CSE 101 | L-1/T-1 | 2022–23]

Q10. Implement a Caesar Cipher in C: shift each character in the string forward by 2 positions (wrapping around). Assume all characters are lowercase letters. Examples: "abc" → "cde", "xyz" → "zab". (12 marks) [CSE 101 | L-1/T-1 | 2022–23]

Q11. Define the string data type. How is a string data type implemented in C? (5 marks) [CSE 101 | L-1/T-1 | 2021–22]

Q12. Take two strings as input. Check whether the first string occurs inside the second one, and if so, output how many times. For example, "desh" is in "Bangladesh" once; "aa" is in "aaa" twice. (8 marks) [CSE 101 | L-1/T-1 | 2021–22]

Q13. Write a program that takes an integer in decimal format from the keyboard and converts it to a string of hexadecimal format. Do not use any built-in function. (10 marks) [CSE 101 | L-1/T-1 | 2021–22]

Q14. Write the following string functions using pointer manipulation (no array indexing):

- (a) `strcpy` — copies one string to another.
- (b) `strcmp` — compares two strings.
- (c) `strcat` — concatenates two strings.

(12 marks)

[CSE 101 | L-1/T-1 | 2021–22]

Q15. Write a C function `char *dbl_dbl(char *str, char ch)` that returns a new string where every single occurrence of character `ch` in `str` is doubled. Double occurrences are left unchanged. Examples: `dbl_dbl("baba", 'a') → "baabaa"`; `dbl_dbl("aa", 'a') → "aaaa"`. (15 marks) [CSE 101 | L-1/T-1 | 2020–21]

Q16. Write a program that searches for the reverse of a given word within a sentence. Both word and sentence are input. If found, print the first position only; otherwise print NOT FOUND. (15 marks) [CSE 101 | L-1/T-1 | 2019–20]

Q17. Write a function that counts the number of words in a string. Words are separated by one or more white-space characters. (15 marks) [CSE 101 | L-1/T-1 | 2019–20]

Q18. Write a function to calculate and return the length of a string passed as a parameter. Do not use `<string.h>`. (5 marks) [CSE 101 | L-1/T-1 | 2018–19]

Q19. Write a program to perform lexicographical comparison between two strings. (10 marks) [CSE 101 | L-1/T-1 | 2018–19]

Q20. Write a program that takes a string (no spaces) followed by a character, separated by a single space. Delete all occurrences of the character from the string. Store the string in a `char` array called `str`; do not declare any other arrays. Do not use `<string.h>`. Example: `str = "abcbabxy", c = 'b' → "acaxy"`. (10 marks) [CSE 101 | L-1/T-1 | 2018–19]

Q21. Write a program that takes 2 strings (may contain spaces) as input and prints their longest common substring. If there are multiple longest common substrings, print any one. For example, for "abcdef" and "ceddefg", the longest common substring is "def". (15 marks) [CSE 101 | L-1/T-1 | 2018–19]

Q22. Write C code to implement the following string functions using only pointer arithmetic (no array indexing):

- (a) `char *strncat(char *dest, char *src, size_t n)` — appends up to `n` characters from `src` to `dest`.
- (b) `char *strchr(char *str, int c)` — finds the first occurrence of character `c` in `str`.
- (c) `int strcmp(char *str1, char *str2)` — compares two strings.
- (d) `int strlen(char *str)` — returns the length of `str`.
- (e) `char *strstr(char *haystack, char *needle)` — finds the first occurrence of `needle` in `haystack`.

(18 marks)

[CSE 101 | L-1/T-1 | 2015–16]

Q23. Write a program that takes an integer (positive or negative) as input and prints it in comma-separated format (a comma after every three digits from the right).

Sample inputs	Sample outputs
-1234567890	-1,234,567,890
-123456	-123,456
-12345	-12,345
-1000	-1,000
-999	-999
-1	-1
0	0
1	1
999	999
1000	1,000
12345	12,345
123456	123,456
1234567890	1,234,567,890

(15 marks)

[CSE 101 | L-1/T-1 | 2015–16]

- Q24.** Write a C function that returns the index of the first occurrence of a substring in a string. Pass the string as the first argument and the substring as the second. Use pointers and avoid array indexing.

```
1 int findSubStrIndex(const char* str, const char* substr);
```

For example `findSubStrIndex("This is a pen", "is")` will return 2, while `findSubStrIndex("This is a pen", "ball")` will return -1. (10 marks)

[CSE 101 | L-1/T-1 | 2015–16]

- Q25.** A substring of a string S is another string S' that occurs in S . For example, "the best of" is a substring of "It was the best of times". Given, 2 strings s and t , you need to detect whether t is a substring of s . Solve this problem by implementing the function declared below. You don't need to write `main()` or any `scanf/printf` statements. Note: You cannot use `string.h` header for this task.

```
1 // return 1 if t is a substring of s
2 // return 0 otherwise
3 int isSubstring(char *s, char *t);
```

(15 marks)

[CSE 105 | L-1/T-2 | 2014–15]

- Q26.** The *longest common substring* problem is to find the longest string that is a substring of two strings. For example, the longest common substring of the strings "ABABC", and "BABABC" is string "BABC" of length 4. Other common substrings are "A", "AB", "B", "BA", "BAB", "ABC", "BC" and "C". Given 2 strings s and t , you need to find the length of their longest common substring. Solve this problem by implementing the function declared below. You don't need to write `main()` or any `scanf/printf` statements. Note: You cannot use `string.h` header for this task. You can use the functions you implemented in answering the previous question.

```
1 //
2 //return the length of longest common substring of s and t
3 //
4 int longestCommonSubstringLength(char *s, char *t);
```

(10 marks)

[CSE 105 | L-1/T-2 | 2014–15]

- Q27.** Write a C function `int strpos(const char *str, char ch)` that finds the first occurrence of character `ch` in string `str` and returns its 0-based position index, or `-1` if not found. (10 marks)

[CSE 105 | L-1/T-2 | 2012–13]

- Q28.** A C program contains the array declaration `char text[100];`. The string "Programming with C can be a challenging creative activity." is assigned to `text`. Show the output of the following `printf()` statements (use `|_` to represent each blank):

- `printf("%18s", text);`
- `printf("%.18s", text);`
- `printf("%18.7s", text);`
- `printf("%-18.7s", text);`

(8 marks)

[CSE 105 | L-1/T-2 | 2011–12]

- Q29.** Write a function `char *reverseword(char *s)` which takes a string as input, detects each word, reverses each word (leaving blanks intact), and returns a pointer to the resultant string. (15 marks)

[CSE 105 | L-1/T-2 | 2010–11]

- Q30.** Write a function `int strrindex(char s[], char t[])` that returns the position of the rightmost occurrence of string `t` in string `s`, or `-1` if there is no such occurrence.

Sample: `s="aabcaabc", t="aa" → 4; s="aabcacacd", t="bc" → 2; s="abcabcabc", t="abcd" → -1.` (12 marks) [CSE 105 | L-1/T-2 | 2010–11]

- Q31.** Implement `char *strtok(char *str, char *ptrn)` with its usual functionality as defined in `<string.h>` using a local static string

variable. (17 marks)

[CSE 101 | L-1/T-2 | 2009–10]

Q32. Write a function `char *strdel(char *s1, char *s2, int n)` that deletes each character from the string `s1` that matches any character in `s2` and appears more than n times in `s1`. The function should return the resultant string while leaving `s1` unchanged. (17 marks)

[CSE 101 | L-1/T-2 | 2009–10]

Q33. Provide definitions for each of the following function declarations. Your code should use pointer arithmetic and avoid array indexing:

- (a) `int strrspn(char *cs, char *ct);` — return the length of the suffix (end) of `cs` consisting of characters in `ct`.
- (b) `int strseq(char *cs, char *ct);` — return 1 if `ct` is a subsequence of `cs`, otherwise return 0. (`ct` is a subsequence of `cs` if all characters of `ct` appear in `cs` in the same relative order.)

(12 marks)

[CSE 101 | L-1/T-2 | 2008–09]

Q34. What will be the output of the following code segment?

```
1 char *p = "ctrl", *q = "alt", c, d;
2 c = *++p;
3 d = ++*q;
4 printf("%c %s\n%c %s", c, p, d, q);
```

(10 marks)

[CSE 101 | L-1/T-2 | 2008–09]

Q35. Write each of the following functions:

- (a) `int strncmp(char *cs, char *ct, int n)` — compare at most n characters of string `cs` to `ct`; return < 0 if `cs` $<$ `ct`, 0 if equal, > 0 if `cs` $>$ `ct` up to n characters.
- (b) `int strspn(char *cs, char *ct)` — return the length of the prefix of `cs` consisting entirely of characters in `ct`.
- (c) `char *strrstr(char *cs, char *ct)` — return a pointer to the last (rightmost) occurrence of string `ct` in `cs`, or NULL if not present.

(18 marks)

[CSE 105 | L-1/T-2 | 2007–08]

Q36. Write a function prototyped as `int strrindex(char s[], char t[])` which returns the position of the rightmost occurrence of `t` in `s`, or -1 if there is none. (10 marks)

[CSE 105 | L-1/T-2 | 2007–08]

Q37. Write a function prototyped as `void insert_str_at(char dest[], char src[], int pos)` which inserts the string `src` into the string `dest` starting at position `pos`. (10 marks)

[CSE 105 | L-1/T-2 | 2007–08]

Q38. Write a function `void str_str(char src[], char substr[], int ith)` which copies the i -th whitespace-separated substring of `src` into `substr`. Whitespace delimiters are `'\n'`, `'\t'`, and `' '`. For example, if `src[] = "abc def efg hij"` and `ith = 2`, then "def" is copied to `substr`. (10 marks)

[CSE 105 | L-1/T-2 | 2007–08]

Q39. Write a C program that will take a sentence as input and print whether it is a palindrome or not. A palindrome is a sentence that reads the same forwards and backwards. (10 marks)

[CSE 105 | L-1/T-2 | 2005–06]

Q40. Implement the following string functions (write down the C code for each function, not a complete program):

- (a) `int strlen(char *str);`
- (b) `char *strcat(char *str1, char *str2);`
- (c) `char *strcpy(char *str1, char *str2);`
- (d) `char *strchr(char *str, char c);`

(20 marks)

[CSE 105 | L-1/T-2 | 2005–06]

Q41. Write a function `void Encrypt(char *s, int key)` that encrypts a string by shifting every letter forward by `key` positions (wrapping around from 'z' back to 'a'). For example, "zany" encrypted with key 4 yields "derc". (10 marks) [CSE 101N | L-1/T-1 | 2003–04]

Q42. Write a program that queries the user for two phrases and indicates whether they are anagrams (every letter appears the same number of times in each phrase). The comparison should be case-insensitive and ignore non-alphabet characters. Assume all inputs are at most 80 characters.

Sample: "Dirty room" and "Dormitory" → anagrams. (12 marks)

[CSE 101N | L-1/T-1 | 2003–04]

Q43. Write a function `void KillSpaces(char *s)` that removes all space characters from a string in-place. The function prototype is given. (7 marks)

[CSE 101N | L-1/T-1 | 2003–04]

Q1. Consider a three-dimensional array with dimensions $l \times m \times n$. An element is indicated by $A[i][j][k]$. What is the equivalent index of this element for a one-dimensional (flattened) array? **(6 marks)** [CSE 101 | L-1/T-1 | 2023–24]

Q2. Define a two-dimensional array to store the marks of m subjects for n students (both n and m taken as input). After reading all marks, output:

- (a) The total marks for each student.
- (b) The highest mark for each subject.

(10 marks)

[CSE 101 | L-1/T-1 | 2023–24]

Q3. Write a program that demonstrates the addition, subtraction, and multiplication of two matrices. Requirements:

- A function to input elements of a matrix.
- The program asks for matrix dimensions and allocates memory dynamically for exactly those dimensions.
- Addition, subtraction, and multiplication functions each create a new resultant matrix.
- A function to free the allocated memory for a matrix.
- A function to print a matrix in a formatted way.
- **You are not allowed to use array declaration or array indexing anywhere in this program.**

(18 marks)

[CSE 101 | L-1/T-1 | 2023–24]

Q4. Write a function in C that takes a square matrix as input and prints:

- (a) The maximum value among the diagonal elements.
- (b) The minimum value among the non-diagonal elements.

(15 marks)

[CSE 101 | L-1/T-1 | 2022–23]

Q5. Write a program that demonstrates addition, subtraction, and multiplication of two matrices. Requirements:

- A function to input matrix elements.
- Ask for dimensions and allocate memory dynamically for exactly those dimensions.
- Each operation function creates a new resultant matrix.
- A function to free the allocated memory.
- A function to print a matrix in a formatted way.

(22 marks)

[CSE 101 | L-1/T-1 | 2021–22]

Q6. Write a complete C program that takes two integer matrices (not necessarily square) as input, multiplies them, and displays the result. Read the sizes and elements from input; use prompt messages and proper indentation. Use the minimum number of variables. **(10 marks)**

[CSE 101 | L-1/T-1 | 2020–21]

Q7. Write a program that counts and prints the number of non-repeated elements located within the two diagonals of an $M \times N$ matrix

A. For the matrix $\begin{pmatrix} 1 & 2 & 1 \\ 3 & 3 & 4 \\ 5 & 6 & 7 \end{pmatrix}$, the answer is 2 (only 5 and 7). **(15 marks)**

[CSE 101 | L-1/T-1 | 2019–20]

Q8. Write a function

```
1 int** mult(int** A, int rA, int cA, int** B, int rB, int cB);
```

that performs matrix multiplication. Dynamically allocate memory for the result and return a pointer to it; return NULL if multiplication is not possible. Also write main to read two matrices from the console using dynamic memory, call `mult`, and print the result.

(20 marks)

[CSE 101 | L-1/T-1 | 2019–20]

Q9. Write code to fill an $N \times N$ matrix $A[N][N]$ with the following pattern (N is input): upper-left triangle filled with 1s, the anti-diagonal with 0s, and the lower-right triangle with -1 s.

1	1	1	1	1	0
1	1	1	1	0	-1
1	1	1	0	-1	-1
1	1	0	-1	-1	-1
1	0	-1	-1	-1	-1
0	-1	-1	-1	-1	-1

(10 marks)

[CSE 101 | L-1/T-1 | 2018–19]

- Q10.** Write a function `int ALTERSUM(int N, int M, int B[][M])` that finds and returns the sum of every alternate element of a 2-D array B (with N rows and M columns), starting from `B[0][0]`. **(13 marks)** [CSE 101 | L-1/T-1 | 2018–19]
- Q11.** A magic square is a square matrix of distinct numbers (each number is used only once), where the numbers in each row, and the numbers in each column, and the numbers in each diagonal, all add up to the same number. For example, the following is a magic square where the numbers in each row, and in each column and, in each diagonal add up to 15.

$$= 15 \qquad = 15$$

6	1	8
7	5	3
2	9	4

$$\begin{array}{l} = 15 \\ = 15 \\ = 15 \end{array}$$

Write a C function `isMagicSquare(int n, int A[][n])` that returns 1 if the $n \times n$ matrix A is a magic square, and 0 otherwise. (13 marks)

[CSE 101 | L-1/T-1 | 2015–16]

- Q12.** Write a C function `getMinOfArray` with proper parameters that takes a three-dimensional array and returns its minimum element. You may add parameters but cannot declare global variables. (7 marks) [CSE 101 | L-1/T-1 | 2015–16]
- Q13.** Define three 100×100 integer arrays. Read the number of rows m and columns n , then read $m \times n$ integers for the first two arrays. Perform matrix multiplication and store the result in the third array. (10 marks) [CSE 101 | L-1/T-1 | 2015–16]
- Q14.** Write a C function to calculate the transpose of a square matrix.

```
1 void transpose(int* list, int n);
2 //list represents the matrix
3 //n is the number of rows/columns
```

(10 marks)

[CSE 101 | L-1/T-1 | 2015–16]

- Q15.** A gray scale image is represented by a 2-dimensional integer array where value of each pixel represented by a number ranges from 0 to 255. A sample gray image is given below:

231	187	178	194	203	190
202	195	198	195	204	192
214	197	179	196	200	197
230	190	180	198	201	187
224	219	193	200	201	186
231	123	189	201	203	197

An error matrix is calculated as follows. At first average of all the pixels are calculated and then error of each pixel is calculated by subtracting the average from the pixel value. Write a C function to calculate error matrix of a given image. Also write a C function to calculate the error frequency list, which represents how many times each error occurs. Arrange the list in ascending order on error values. **(15 marks)**

[CSE 101 | L-1/T-1 | 2015–16]

- Q16.** Implement a function `diagonalsSum` that returns the sum of the elements of both diagonals of an $n \times n$ matrix. Design the parameter list appropriately. Also write a `main` function to read n ($1 \leq n \leq 10$) and the matrix, then call `diagonalsSum` and print the result. Do not use global variables. **(15 marks)** [CSE 105 | L-1/T-2 | 2014–15]
- Q17.** Write a program to sort all elements in a 4×4 matrix row-wise.

$$\text{Input: } \begin{bmatrix} 4 & 3 & 2 & 1 \\ 4 & 3 & 3 & 1 \\ 4 & 3 & 2 & 1 \\ 4 & 3 & 2 & 2 \end{bmatrix} \qquad \text{Output: } \begin{bmatrix} 1 & 2 & 3 & 4 \\ 1 & 3 & 3 & 4 \\ 1 & 2 & 3 & 4 \\ 2 & 2 & 3 & 4 \end{bmatrix}$$

(15 marks)

[CSE 105 | L-1/T-2 | 2010–11]

- Q18.** Consider the declaration `int A[10][10];`. Redefine the declaration using only pointer and dynamic memory allocation strategies. Assume that two other matrices B and C are declared the same way, with values in A and B. Write a function that computes the product of matrices A and B and stores the result in C **without** using any bracket notation (`[]`). (15 marks) [CSE 101 | L-1/T-2 | 2009–10]
- Q19.** Explain the three ways in which a two-dimensional data structure can be created in C. Provide a memory representation diagram and memory allocation code (if any) for each case. (14 marks) [CSE 101 | L-1/T-2 | 2008–09]

Q20. Consider the two code fragments:

```
1  /* Fragment 1 */
2  char s1[][7] = {"CSE101", "CSE102"};
3
4  /* Fragment 2 */
5  char *s2[] = {"CSE101", "CSE102"};
```

Which code fragment has better memory utilisation and how? (5 marks)

[CSE 101N | L-1/T-1 | 2003–04]

Q21. A 0/1 matrix is called a **diagonal matrix** if its two main diagonals contain only 1s. Write a program to determine whether a given $n \times n$ integer matrix M is diagonal or not.

1	0	1
0	1	1
1	1	1

0	0	1
1	1	1
1	0	1

Diagonal matrix

Non-Diagonal matrix

(8 marks)

[CSE 101N | L-1/T-1 | 2003–04]

BUET · CSE 101 · Structured Programming

Part III

Pointers & User-Defined Types

Pointers & User-Defined Types

S5 — Pointers

Pointer Basics

Q1. A union is an administrative unit under an upazilla, an upazilla is an administrative unit under a district and a district is an administrative unit under a division. You are to store the temperature of a union so that it can be accessible by the division, district, upazilla and union. Write necessary programs to implement this by using array or pointer. Demonstrate how a temperature of a particular union can be accessed. (9 marks) [CSE 101 | L-1/T-1 | 2024–25]

Q2. Consider the following code which uses arrays and pointers. Is there any compilation error? Find the output:

```
1 int a[10];
2 for (int i=0; i<10; i++) a[i]=i;
3 int *p;
4 p = a+5;
5 for (int i=3; i<8; i++) printf("\n%d", *(p+i));
6 p = p-3;
7 for (int i=0; i<10; i++) {
8     printf("\n%d", *p);
9     p++;
10 }
```

(5 marks)

[CSE 101 | L-1/T-1 | 2023–24]

Q3. Analyse the code below and find all bugs with proper explanation:

```
1 #include <stdio.h>
2
3 int* square(int x) {
4     int result = x * x;
5     return &result;
6 }
7
8 int main() {
9     int a = 10;
10    int *p;
11    p = square(a);
12    printf("%d", *p);
13    return 0;
14 }
```

(11 marks)

[CSE 101 | L-1/T-1 | 2022–23]

Q4. See the given code below (assume the value of p is 100 initially):

```
1 #include <stdio.h>
2 int main() {
3     int arr[6] = {1, 2, 3, 4, 5};
4     int *p = arr; // assume p = 100
5
6     printf("%d\n", p);
7     printf("%d\n", *p);
8     printf("%d\n", *(p+2));
9     printf("%d\n", p++);
10    printf("%d\n", *p);
11    return 0;
12 }
```

Write down the output of this program. (8 marks)

[CSE 101 | L-1/T-1 | 2022–23]

Q5. Consider the following program with the stack frame given. Answer:

```
1 #include <stdio.h>
2
3 void swap(int *a, int *b)
4 {
5     int temp = *a;
6     *a = *b;
7     *b = temp;
8 }
9
```

```

10 void swap_2(int ***a, int ***b)
11 {
12     int **temp = *a;
13     *a = *b;
14     *b = temp;
15 }
16
17 int main() {
18     int p = 25, q = 50;
19     int* a = &p;
20     int* b = &q;
21     int **c = &a;
22     int **d = &b;
23     int ***e = &c;
24     int ***f = &d;
25
26     swap(**e, **f);
27     printf("%d %d %d %d\n", p, q, *a, *b);
28     printf("%d %d %d %d %d %d\n", a, b, c, d, e, f);
29     printf("%d %d %d %d\n", **e, ***e, **f, ***f);
30
31     swap_2(e, f);
32     printf("%d %d %d %d\n", p, q, *a, *b);
33     printf("%d %d %d %d %d %d\n", a, b, c, d, e, f);
34     printf("%d %d %d %d\n", **e, ***e, **f, ***f);
35     // Note: All the format specifiers are intentionally in %d
36     // You can safely assume that the 8 bytes pointer values
37     // can be converted into 4 bytes int values
38     return 0;
39 }

```

Stack addresses: p=1000, q=1004, a=1008, b=1016, c=1024, d=1032, e=1040, f=1048.

- Briefly explain the effects of lines 26 and 31.
- Write the output of lines 27, 28, and 29.
- Write the output of lines 32, 33, and 34.
- Modify the body of `swap_2` so that replacing line 26 with `swap_2(e, f)` produces the same effect as `swap(**e, **f)`. Use dynamic allocation for the temp variable and avoid memory leaks.

(23 marks)

[CSE 101 | L-1/T-1 | 2021–22]

Q6. Describe the output of the following code segment. Assume `int` = 2 bytes, `float` = 4 bytes. Variables `a`, `b`, `x`, `y` start at addresses F9C, F9E, A130, B2C3 respectively.

```

1 int a = 8, b, *pa, *pb;
2 float x = 0.001, y = 0.2, *px = &x, *py = &y;
3 pb = &a;
4 b = *pb + 7;
5 pa = pb;
6 *px *= *pa + *pb;
7 *py += *pa;
8 printf("%x %x %d %.2f", pa + 2, &(*pa), a, x);

```

(8 marks)

[CSE 101 | L-1/T-1 | 2020–21]

Q7. Assume you have an integer array called `numbers` with `n` integers. Write a code snippet to check whether the array is sorted (either ascending or descending). You cannot use array indexing; use pointer arithmetic and pointer dereferencing instead. (10 marks)

[CSE 101 | L-1/T-1 | 2019–20]

Q8. Consider the following program fragment:

```

1 int a, b;
2 int *p, *q;
3 p = &a;
4 q = &b;
5 scanf("%d%d", &a, &b);
6 printf("a=%d b=%d", a, b);

```

Rewrite the `scanf` and `printf` statements using only the pointer variables `p` and `q`. (5 marks) [CSE 101 | L-1/T-1 | 2015–16]

Q9. Consider the following function:

```

1 int *f() {
2     int a[20] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
3     return a;
4 }

```

What is the problem with this function? Describe two different ways to fix it. (7 marks)

[CSE 101 | L-1/T-1 | 2015–16]

Q10. Explain the declaration of the following function prototype:

```

1 char (*( *funct [])(int (*a) [])) [] ;

```

(8 marks)

[CSE 101 | L-1/T-1 | 2015–16]

Q11. A C program contains the following statements:

```

1 char u, v = 'A';
2 char *pu, *pv = &v;
3 *pv = v + 1;
4 u = *pv + 1;
5 pu = pv;
6 pu = &u;

```

Each character occupies 1 byte. If u is stored at address F8C and v at address F8D, answer:

- Which statement is invalid?
- What value is represented by &v?
- What value is assigned to pv?
- What value is represented by *pv?
- What value is represented by *pu?

(10 marks)

[CSE 105 | L-1/T-2 | 2012–13]

Q12. Consider the following code fragment. Assume x and p are correctly declared as an integer and a pointer to integer respectively. Explain what the following loop does:

```

1 p = &x; x = 0;
2 while (*p == x) { *p = *p + 1; }

```

(3 marks)

[CSE 105 | L-1/T-2 | 2011–12]

Q13. Your friend wrote the following swap() function. He needs to swap two variables and calls it from main() with arguments a and b. Unfortunately it does not work. Explain why, and correct it:

```

1 void swap(int x, int y) {
2     int temp;
3     temp = x;
4     x = y;
5     y = temp;
6 }

```

(6 marks)

[CSE 105 | L-1/T-2 | 2011–12]

Q14. Consider the following program where Output Line #1 gives 22FF24. What will be written in Output Lines #2 to #9?

```

1 char *pChar; int *pInt;
2 char aChar[] = {0x78,0x56,0x34,0x12,0x12,0x34,0x56,0x78};
3 printf("%X\n", aChar); /* Line 1 */
4 pChar = aChar;
5 pInt = (int *)aChar;
6 printf("%X\n", pChar); /* Line 2 */
7 printf("%X\n", *pChar); /* Line 3 */
8 pChar++;
9 printf("%X\n", pChar); /* Line 4 */
10 printf("%X\n", *pChar); /* Line 5 */
11 printf("%X\n", pInt); /* Line 6 */
12 printf("%X\n", *pInt); /* Line 7 */
13 pInt++;
14 printf("%X\n", pInt); /* Line 8 */
15 printf("%X\n", *pInt); /* Line 9 */

```

(5 marks)

[CSE 105 | L-1/T-2 | 2011–12]

Q15. Explain the following complicated declaration:

```
1 char (*(x())[5])();
```

(5 marks)

[CSE 105 | L-1/T-2 | 2011–12]

Q16. Describe the output of the following code segment:

```
1 char *p = "ctrl", *q = "alt", c, d;
2 c = *++p;
3 d = **q;
4 printf("%c %s\n%c %s", c, p, d, q);
```

(10 marks)

[CSE 101 | L-1/T-2 | 2009–10]

Q17. Mention the differences between the following declarations:

```
1 char am[] = "the pond is big";
2 char *pm = "the pond is big";
```

(10 marks)

[CSE 105 | L-1/T-2 | 2007–08]

Q18. Explain the following statements and write the corresponding outputs:

```
1 int main(){
2     char array[]={ '1', '5', '9', '7', '3' };
3     char* x,y;
4     x=array+1;
5     y=*array+1;
6     printf("%c %c",*x,y);
7     y=*x;
8     x++;
9     printf("%c %c\n",*x,y);
10    y=*(array+4);
11    x=&y;
12    printf("%c %c\n",*x,y);
13    return 0;
14 }
```

(12 marks)

[CSE 105 | L-1/T-2 | 2005–06]

Q19. Identify the problems in the following code fragments:

```
1 int *p;
2 *p = 20;
```

```
1 int x = 10, *p = &x, q;
2 p = q;
```

(b) The macro `#define square(x) x*x` is invoked with `square(z+1)`.

(6 marks)

[CSE 101N | L-1/T-1 | 2003–04]

Pointers to Arrays, Strings, Structs & Functions

(c) The trades of a particular day is defined by $(n_1, p_1), (n_2, p_2), \dots, (n_i, p_i), \dots, (n_m, p_m)$. Here n_i indicates the amount of i -th trade and p_i indicates the price of the i -th trade. The average rate of the trade is $(n_1 \times p_1 + n_2 \times p_2 + \dots + n_i \times p_i + \dots + n_m \times p_m) / (n_1 + n_2 + \dots + n_i + \dots + n_m)$. Write the following functions:

- The function takes the number of trades m as parameter, facilitates taking inputs for the trades and returns a pointer that points to the stored elements for the trades.
- The function takes a pointer storing a series of trades as parameter and returns the average of the trades.
- The function takes two pointers storing a series of trades as parameters and returns the average of the combined series of trades.

Write the main function so that it takes two series of trades and shows the average rates. Then determine the combined average rates of the two trade series. (18 marks)

[CSE 101 | L-1/T-1 | 2024–25]

- Q1.** Write a function that takes a **string pointer** and reverses the string **in-place**. It is recommended not to use any other string or array inside the function. **(8 marks)** [CSE 101 | L-1/T-1 | 2023–24]
- Q2.** Write the functions `process_char_vowel`, `process_char_cons`, and `count_char_types` so that the following `main` correctly prints the total vowels and consonants in the string "I am confident I will pass this exam!!!":

```
1 int main() {
2     char *str = "I am confident I will pass this exam!!!";
3     int (*callback_ptr_vowel)(char) = process_char_vowel;
4     int (*callback_ptr_cons)(char) = process_char_cons;
5     int total_vowels = count_char_types(str, callback_ptr_vowel);
6     int total_cons = count_char_types(str, callback_ptr_cons);
7     printf("%d\n", total_vowels);
8     printf("%d\n", total_cons);
9     return 0;
10 }
```

- You cannot change `main`; only write the required functions. **(15 marks)** [CSE 101 | L-1/T-1 | 2021–22]
- Q3.** Consider the following code using arrays and pointers. Find the output:

```
1 int a[10];
2 for (int i = 0; i < 10; i++) a[i] = i;
3 int *p;
4 p = a + 5;
5 for (int i = 0; i < 5; i++) printf("\n%d", *(p+i));
6 p = a + 2;
7 for (int i = 0; i < 10; i++) {
8     printf("\n%d", *p);
9     p++;
10 }
```

- (4 marks)** [CSE 101 | L-1/T-1 | 2021–22]
- Q4.** Consider the declaration

```
1 int p[2][3][2] = {1,2,3,4,5,6,7,8,9,10,11,12};
```

- If memory location `p` contains a value of `B040` (in hexadecimal) ; an integer is 2 bytes. Show the address of each integer. Write down the values of:
- (a) `**p`
 - (b) `*(*p+1)+2`
 - (c) `**(*p+2)`
 - (d) `*(*p+1)+1`
- (10 marks)** [CSE 101 | L-1/T-1 | 2020–21]
- Q5.** Consider the declaration `int p[3][4] = {1,2,3,4,5,6,7,8,9,10,11,12};`. The first element is at address 12040; an integer is 2 bytes. Show the address of each integer. Write down the values of:
- (a) `**p`
 - (b) `*(*p+1)+2`
 - (c) `**(*p+2)`
 - (d) `*(*p+1)+1`
- (10 marks)** [CSE 101 | L-1/T-1 | 2015–16]

- Q6.** Consider the following declaration:

```
1 static float table[2][3][2] = {
2     {{1.1,1.2},{1.3,1.4},{1.5,1.6}},
3     {{2.1,2.2},{2.3,2.4},{2.5,2.6}}
4 };
```

Floating values are stored at memory addresses 100–116 (2 bytes per float). Fill in the values of:

0 th 2D Array						1 st 2D Array					
1.1	1.2	1.3	1.4	1.5	1.6	2.1	2.2	2.3	2.4	2.5	2.6
100	102	104	106	108	10A	10C	10E	110	112	114	116

- (a) `*(table+1)`
- (b) `*((*table+1))+1`
- (c) `**(*table+1)`
- (d) `***table+1`
- (e) `*(*table+1)+1`

(10 marks)

[CSE 105 | L-1/T-2 | 2013–14]

- Q7. (a) Write an appropriate declaration for: *“a pointer to a function that accepts an argument which is an array of pointers to integer quantities and returns a pointer to a character”*.
- (b) Explain the purpose of the following declaration: `float (*x[20])(int *a);`

(10 marks)

[CSE 105 | L-1/T-2 | 2013–14]

- Q8. Explain the following declaration with necessary memory-representation figure(s):

```
1 int (*(*x[3])())[5];
```

(10 marks)

[CSE 101 | L-1/T-2 | 2008–09]

- Q9. Write down the C declaration for: *x as a 3-element array of pointer to function returning pointer to a 5-element array of char.* (7 marks)

[CSE 105 | L-1/T-2 | 2007–08]

- Q10. Explain the operation of the following function. Rewrite this function using a pointer.

```
1 void test(char* s1, char* s2){
2     int i=0, j;
3     while(s1[i]!='\0'){
4         i++;
5     }
6     for(j=0; s1[j]; i++, j++){
7         /* */
8     }
9 }
```

(14 marks)

[CSE 105 | L-1/T-2 | 2005–06]

- Q11. State a case in which a pointer to function is useful. Write the appropriate declaration for each of the following:

- (a) Declare a function that accepts another function as an argument and returns a pointer to `char`. The argument function accepts an `int` and returns an `int`.
- (b) Declare a pointer to a function that accepts three `int` pointers as arguments and returns a pointer to `float`.

(8 marks)

[CSE 101N | L-1/T-1 | 2003–04]

- Q12. Rewrite the following code using a **pointer to function** (the target function is `strcmp`):

```
1 char s1[] = "Here", s2[] = "here";
2 if (strcmp(s1, s2) == 0)
3     printf("Two strings are same");
4 else
5     printf("They differ.");
```

(7 marks)

[CSE 101N | L-1/T-1 | 2003–04]

- Q13. Let a structure be defined as:

```
1 struct String {
2     char str[10];
3     int len;
4 };
5 struct String s[] = {"ab", 2}, {"def", 3};
6 struct String *p;
7 p = s;
```

What do the following expressions mean?

- (a) `(p->str)++`
- (b) `++p->str`
- (c) `*(p->str)`

(5 marks)

[CSE 101N | L-1/T-1 | 2003–04]

Dynamic Memory Allocation

Q1. Why is dynamic memory allocation necessary? What are the measures to be taken to prevent memory leakage during memory allocation? Explain with examples. (8 marks) [CSE 101 | L-1/T-1 | 2024–25]

Q2. How can you perform memory allocation in C? What is memory leakage? How can you prevent memory leakage when memory is allocated inside a function but cannot be freed there (because it will be used by other functions)? Explain with examples. (10 marks) [CSE 101 | L-1/T-1 | 2023–24]

Q3. Explain two different ways to implement a **three-dimensional array using pointers** in C, with necessary examples. (7 marks) [CSE 101 | L-1/T-1 | 2023–24]

Q4. Create a C program that inputs two matrices, multiplies them, and displays the result. Your program must contain the following functions:

```
1 int** takeMatrix(int rows, int columns);
2 void printMatrix(int **array, int rows, int cols);
3 int** multiplyMatrix(int **a, int **b,
4                     int rowsA, int colsA,
5                     int rowsB, int colsB);
6 void freeMatrix(int **matrix, int rows);
```

Use dynamic memory allocation and free all allocated storage. (20 marks)

[CSE 101 | L-1/T-1 | 2022–23]

Q5. Re-declare `double A[10][20][30]` using only pointers and `malloc()`. Do not use the `[]` operator. (5 marks) [CSE 101 | L-1/T-1 | 2020–21]

Q6. Suppose a variable for storing a list of country names is declared as `char *countries[]`; . Write a C loop that reads country names until the word "end" is typed, allocating necessary memory. Also write a function `void reorder(int n, char *list[])` that sorts the list in dictionary order. (12 marks)

[CSE 101 | L-1/T-1 | 2020–21]

Q7. Consider the following code. Is it possible that this code might fail to run? Why or why not? How can you prevent this from happening?

```
1 #include <memory.h>
2 int funny(int a, int b) {
3     char *hog = malloc(1024*1024*1024*50);
4     return a + b;
5 }
6 int main(void) {
7     int sum = funny(funny(6, 8), funny(3, 7));
8     return 0;
9 }
```

(15 marks)

[CSE 101 | L-1/T-1 | 2020–21]

Q8. Why do we need dynamic memory allocation? (5 marks)

[CSE 101 | L-1/T-1 | 2020–21]

Q9. Write a program that reads the heights (in cm) of several students. First read the count n , then n real values. Use dynamic memory allocation. After storing heights, find and print the difference between the tallest and shortest student. Free the allocated memory. (10 marks) [CSE 101 | L-1/T-1 | 2018–19]

Q10. Using a pointer to pointer and dynamic memory allocation, write a program to store a matrix and print the average of integers in each column. Input: first two integers are number of rows r and columns c ; then the matrix in row-major order. Free all dynamically allocated memory before exit. (15 marks) [CSE 101 | L-1/T-1 | 2018–19]

Q11. Consider the following C library functions

- `void *malloc (int size)` – allocates the requested memory and returns a pointer to it. It doesn't initialize the allocated memory.
- `void *realloc (void *ptr, int size)` – attempts to resize the memory pointed to by `ptr` that was previously allocated using `malloc`. It is used to fill memory.
- `void *memset (void * ptr, int value, int num)` – sets the first `num` bytes of memory pointed by `ptr` to the specified value. It is used to fill memory.

- `void * memcpy (void * destination, void * source, int num)` – copies the values of num bytes from the location pointed to by source directly to the memory pointed by destination. It is used to copy memory.

Now do the following using malloc, memset and memcpy (you can't use realloc)

- Write C code to make your own function `void *mymalloc (int size)` that allocates the requested memory and initializes it to zero.
- Write C code to make your own function `void * myrealloc (void *oldptr, int oldSize, int newSize)` that implements the realloc library function.

(8 marks)

[CSE 101 | L-1/T-1 | 2015–16]

Q12. Write two different C programs that dynamically allocate memory for the following triangular array (level i has i elements):

1							
1	1	1					
1	1	1	1	1			
1	1	1	1	1	1	1	
1	1	1	1	1			
1	1	1					
1							

- Using an array of pointers.
- Using a pointer to pointer.

Free all dynamically allocated memory at the end of each program. (14 marks)

[CSE 101 | L-1/T-1 | 2015–16]

Q13. Re-declare `double A[20][36][37]` using only pointers and `malloc()`. Do not use the `[]` operator. (7 marks) [CSE 101 | L-1/T-1 | 2015–16]

Q14. Rewrite the following declaration using dynamic memory allocation to allocate an equivalent amount of memory:

```
1 int matrix[NoRow][NoCol];
```

(5 marks)

[CSE 105 | L-1/T-2 | 2012–13]

Q15. Consider a linked list based on the following structure:

```
1 typedef struct { int month, day, year; } date;
2 typedef struct xx {
3     char empID[10]; char name[30];
4     char street[40]; char city[20];
5     date birthdate; int salary;
6     struct xx *next;
7 } record;
```

The first element is `start` and the last element is `NULL`. Write:

- A C function to display `empID`, `name`, and `city` of all employees whose salary is ≥ 20000 , in tabular form.
- A C function to delete all employees from the list whose `city` is "Khulna".

(20 marks)

[CSE 105 | L-1/T-2 | 2012–13]

Q16. Write a C program to read n integers into an array and split it into two dynamically allocated arrays: one for even numbers and one for odd numbers. Allocate only as much memory as needed (no wasted memory).

Example: Original: 7,9,4,6,5,3,2,10,18 $\rightarrow$ Odd: 7,9,5,3; Even: 4,6,2,10,18. (15 marks) [CSE 105 | L-1/T-2 | 2010–11]

Q17. The following function is supposed to allocate memory for an integer array. Does it successfully allocate memory for variable `p` in `main`? Justify your answer.

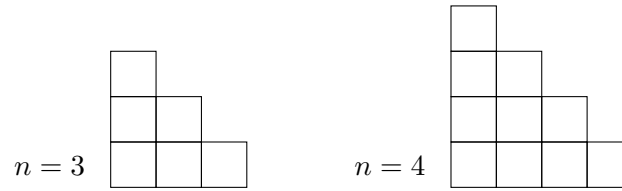
```
1 #include <stdlib.h>
2 void allocate(int t, int size) {
3     t = (int)malloc(sizeof(int) * size);
4 }
5 int main() {
6     int *p;
7     allocate(p, 100);
8 }
```

(7 marks)

[CSE 105 | L-1/T-2 | 2010–11]

Q18. Declare an array of 5 pointers to characters. Allocate exactly enough memory to store the strings "Bangladesh", "India", "SriLanka", "Nepal", and "Bhutan" (no extra memory). Show how you store the strings in the array. **(10 marks)** [CSE 105 | L-1/T-2 | 2010–11]

Q19. (a) Write a program that defines a triangular array with n levels where level i contains i elements ($i = 1, 2, \dots, n$). The value of n is received as a command-line argument. Implement the array using pointer and dynamic memory allocation.



(7 marks)

[CSE 101 | L-1/T-2 | 2009–10]

(b) Write an insertion function for the triangular array above which inserts elements so that they become row- and column-wise sorted. **(7 marks)** [CSE 101 | L-1/T-2 | 2009–10]

(c) Write an efficient search function that searches an element in the triangular array and returns the row and column position as a double in the format row.column if present, or -1.0 if absent. **(7 marks)** [CSE 101 | L-1/T-2 | 2009–10]

(d) Write a fetch function that retrieves a value from a given row and column position in the triangular array, or returns -1.0 if the position is invalid. **(7 marks)** [CSE 101 | L-1/T-2 | 2009–10]

Q20. Consider the following declarations (doubly linked list):

```
1 typedef struct { int month, day, year; } date;
2 typedef struct xx {
3     char name[20]; char street[30]; char city[20];
4     int acct_no; char acct_type;
5     float oldbalance, newbalance, payment;
6     date lastpayment;
7     struct xx *next, *prev;
8 } record;
```

A doubly linked list has already been created. The first element is named **start**; the **prev** of **start** and the last element's **next** are both NULL. Write a C function to reorganize the records so that the list becomes sorted in ascending order of **acct_no**. **(25 marks)** [CSE 101 | L-1/T-2 | 2009–10]

Write a C function to insert a new record in the sorted doubly linked list obtained from the previous question. **(10 marks)** [CSE 101 | L-1/T-2 | 2009–10]

Q21. What is the purpose of using pointers in a C program? Consider the declaration `int **x`; Write necessary C code for memory allocation and initialization of **x** such that:

$$x[i][j] = \frac{2i}{3} + j(j-1) \quad \text{for } 0 \leq i, j \leq 10$$

(10 marks)

[CSE 101 | L-1/T-2 | 2008–09]

Q22. Consider the declaration `int **pa`; Write necessary C code for memory allocation and initialization of **pa** such that $pa[i][j] = i*(i+1) + j/2$ for $0 \leq i \leq 6$ and $0 \leq j \leq 5$. **(8 marks)** [CSE 105 | L-1/T-2 | 2007–08]

Q23. Consider the following declaration:

```
1 int **p;
```

Write C code for dynamic memory allocation so that **p** can point to a 10×20 table of integers (10 rows, 20 columns). **(5 marks)** [CSE 105 | L-1/T-2 | 2005–06]

Q24. What is the problem with the following code fragments?

```
1 char *s;
2 s = (char *) malloc(100);
3 s = "hello";
4 free(s);
```

```
1 FILE *fp = fopen("file.dot", "r");
2 while (ch = getc(fp) != EOF)
3     putc(stdout, ch);
```

(5 marks)

[CSE 101N | L-1/T-1 | 2003–04]

Q25. How can you know whether `malloc()` was successful in allocating memory? What is the purpose of the stack area in RAM? (5 marks) [CSE 101N | L-1/T-1 | 2003–04]

Q26. Look at the following code where a function dynamically allocates memory for a pointer:

```
1 void allocate(int *p, int n) {
2     p = (int *) malloc(sizeof(int) * n);
3     if (p == NULL) exit(0);
4 }
5 void main() {
6     int *pa;
7     allocate(pa, 10);
8     pa[2] = 4;
9 }
```

What is the potential problem of using `allocate()` for pointer `pa`? Rewrite the function to fix the problem. (8 marks) [CSE 101N | L-1/T-1 | 2003–04]

S6 — User-Defined Data Types

Structures

Q1. A two dimensional point has x and y . Design a structure for that. Next, write a function that takes two variables of the structure type and returns the distance between the two points. Declare another structure with three members of the point structure type to store triangles. Next, write another function that takes a triangle structure as its parameter, and return the area of the triangle. You will use the pass by reference method to pass all structures to the respective functions. (12 marks) [CSE 101 | L-1/T-1 | 2024–25]

Q2. Consider a `struct studentstruct` defined with several fields (including at least one pointer or array field). Given:

```
1 struct studentstruct { /* ... */ };
2 struct studentstruct a, b;
3 /* Taking input into a */
4 b = a; /* Copying a to b */
```

What problems may arise from such a direct struct assignment? What is the solution to make the copy correct (deep copy)? (8 marks) [CSE 101 | L-1/T-1 | 2023–24]

Q3. Using a C program, define a structure named `Student` that includes attributes ID, name, and CGPA. Input an array of students from the user, sort them based on CGPA, and display the sorted list. (12 marks) [CSE 101 | L-1/T-1 | 2022–23]

Q4. Implement the functions `createAccount`, `searchAccount`, `addBalance`, `transferBalance`, and `printTransactions` for a banking system with the following structure definitions:

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <time.h>
4
5 #define TOTAL_ACCOUNTS_AT_MOST 1000
6 #define TOTAL_TRANSACTIONS_AT_MOST 10000
7
8 typedef enum {
9     SAVINGS,
10    CURRENT
11 } AccountType;
12
13 typedef enum {
14     WITHDRAW,
15     DEPOSIT,
16     TRANSFER
17 } TransactionType;
18
19 typedef struct {
20     int date;
21     int month;
22     int year;
23 } Date;
24
25 typedef struct {
26     Date date;
```

```

27     int hour;
28     int mins;
29     int seconds;
30 } Time;
31
32 typedef struct {
33     char *name;
34     int accountNumber;
35     int balance;
36     AccountType type;
37     Date dob;
38     Time openingTime;
39 } Account;
40
41 typedef struct {
42     Account *sentFrom;
43     Account *sentTo;
44     Time time;
45     int amount;
46     TransactionType type;
47 } Transaction;
48
49 Account* allAccounts[TOTAL_ACCOUNTS_AT_MOST];
50 Transaction* allTransactions[TOTAL_TRANSACTIONS_AT_MOST];
51
52 Time getCurrentTime();
53
54 // create an account and add the account to allAccounts array
55 // set account opening time by calling the function getCurrentTime
56 Account *createAccount(char *name, int accountNumber, int initialBalance, AccountType type, Date dob);
57
58 // search for the account number given.
59 Account *searchAccount(int accountNumber);
60
61 // add balance to an accountNumber. Add a transaction to the allTrasancctions array
62 void addBalance(int account, int amount);
63
64 // withdraw balance. Add a transaction to the allTrasancctions array
65 void withdrawBalance(int account, int amount);
66
67 // transfer balance from one account to another account. Add a transaction to the allTrasancctions
68 // array
69 void transferBalance(int from, int to, int amount);
70
71 // print all transactions of an account
72 void printTransactions(int account);

```

You cannot change any function prototypes. (25 marks)

[CSE 101 | L-1/T-1 | 2021–22]

Q5. Look at the following C code:

```

1 struct X { short s; int i; char c; };
2 struct Y { int i; short s; char c; };
3 struct Z { int i; char c; short s; };
4 const int sizeX = sizeof(struct X);
5 const int sizeY = sizeof(struct Y);
6 const int sizeZ = sizeof(struct Z);
7 int main() {
8     printf("%d %d %d\n", sizeX, sizeY, sizeZ);
9     return 0;
10 }

```

Write the output and provide a brief justification for each of `sizeX`, `sizeY`, and `sizeZ`. Assume `int`=4 bytes, `char`=1 byte, `short`=2 bytes. (15 marks)

[CSE 101 | L-1/T-1 | 2020–21]

Q6. Define a structure `District` to store year-long weather data about a district of Bangladesh. It should contain:

```

1 Id (Integer identifier of district, between 1 and 64),
2 Temperature (An array of 365 real numbers),
3 Humidity (array of 365 real numbers),
4 Rain (An array of 365 real numbers).

```

Declare an array named `districts` of type `District`. (10 marks)

[CSE 101 | L-1/T-1 | 2019–20]

Q7. Suppose we have a list of student names, identification numbers, and grades. For example, the beginning of the list might look like:

Casanova	910017	B
Ayaan	934422	A
Smith	978766	C

Suppose there are five possible grades A, B, C, D, and F.

- (i) Write down a program called **recorder** that will perform the following tasks:

Take the above data as input and put it into an array of structure. The number of students N will also be input to your program first. Define your own structure. Assume that the names are single word names with max length of 50. The program should print out an ordered list of students and grades, i.e., students with A grades should be listed first, students with B grade next and so forth. Among all students having the same grade, the students should be listed alphabetically by name.

- (ii) Add a function **class_average** to your program that will compute the class average and print it. Assume that A grade has value 4.0, a B grade has value 3.0, and so forth.

(20 marks)

[CSE 101 | L-1/T-1 | 2018–19]

Q8. Which of the two following structures consumes less memory? Show the calculation. Assume **int**=4 bytes, **char**=1 byte, **float**=4 bytes.

```
1 struct A { char a; float d; char c; };
2 struct B { char a; char c; float d; };
```

(5 marks)

[CSE 101 | L-1/T-1 | 2018–19]

Q9. The English Premier League (EPL) consists of 20 teams. Each team has a name, number of matches played, number of matches won, number of matches drawn, number of matches lost, total goals scored and total goals conceded.

A sample data for 6 team is given below:

Name	Played	Win	Draw	Loss	Goals Scored	Goals Conceded
Manchester United	38	23	11	4	78	37
Chelsea	38	21	9	8	69	33
Manchester City	38	17	7	14	59	44
Arsenal	38	19	11	8	72	43
Liverpool	38	16	14	8	55	46
Tottenham Hotspur	38	21	8	9	60	33

Now do the following:

- (i) Define a C structure for a EPL team and necessary variables to work with 20 EPL teams.
- (ii) Write a function **loadFromFile** that loads all the club's information from a file **clubs.txt** to your program.
- (iii) Each club will get 3 points for a win, 1 point for a draw and no point for a loss. Based on this point scheme, write a function **printPosition** that prints each team's total point and their position. For example, the output for the above sample of 6 clubs will be as follows:

Name	Total Points	Position
Manchester United	80	1
Chelsea	72	2
Manchester City	58	6
Arsenal	68	4
Tottenham Hotspur	71	3
Liverpool	62	5

- (iv) Write a function **storeToFile** that sorts the above information based on ascending order of position and write to a file named **positions.txt**.

Note that, You can add parameters to the functions if necessary but you cannot hard code based on the sample data. (18 marks)

[CSE 101 | L-1/T-1 | 2015–16]

Q10. A hall management system records information of all students who are either resident or attached to that hall. The record has the following fields:

Fields	Description
name	Name of the student
stdID	Student number
DoB	Death of birth of the student
enrolDate	Date of admission into the hall
Room No	Room number of the residential student
GPA	An array contains semester-wise GPA

- (a) Write down a suitable data structure to store the above mentioned information of all students associated with the hall. **(10 marks)**
- (b) Assume that `studentList` is a pointer to the list of all students as declared below:
`Student *StudentList;`
 Write down a C function to list all the students (containing name, stdID and roomNo) whose CGPA is less than 3.0 and occupied seats in the hall. **(10 marks)**
 Write a C function that will retrieve information of all students stored in the hard-disk into the variable `studentList` as declared in (b). At the end of the program, you are required to replace the information stored in the hard-disk with the information recorded in `studentList`, which is referred as **save** operation. Write another C function for **save** operation. Both of the function must be written in such a way that a student's information is considered as a single entity. (Hints: You are not allowed to use `fscanf()` or `fprintf()` functions in your program.) **((8+7) marks)**

[CSE 101 | L-1/T-1 | 2015–16]

Q11. You need to write some code for a banking system. In this system, there is a structure called *AccountBalance* to hold the account id (integer) and balance (double precision floating point). The structure is used to read account balance information from file and store in memory as needed, and vice versa. The total number of accounts is defined in a constant *N*. The accounts are identified by integers from 1 to *N*.

Today's opening balance information of all accounts is stored in a binary file called "dayOpeningBalance.dat". The file contains the data of the accounts in increasing order of account id. For each account, it stores an *AccountBalance* structure. Another binary file, called "currentBalance.dat" has the same format as the former file, but contains the current balance (instead of opening balance) for each account. Additionally, each transaction is logged into a text file called "transactions.log". In this file, each line contains an account id, followed by an amount (double precision floating point) that was added (positive number) to or deducted from (negative number) the account in a transaction. Obviously, there can be multiple transactions for an account in the log file.

- Write the definition of *AccountBalance* structure. What would be the size of it in bytes?
- Write a program that reads from console an account id, and outputs the opening and current balance of the corresponding account. For efficiency, you *must* use random access, instead of sequential access of the files. You may assume that the input account id is valid.
- In the above scenario, instead of reading the account id from console, read it as a command line argument and write necessary code to obtain the account id as an integer. (You don't need to rewrite the parts of reading/printing the balances)
- Write a program that processes all the recorded transactions to check that the current balance of each account is consistent, For example, if opening balance of account# 1 is 4000 and there are 2 transactions in the log for account# 1 which are "1 2000" and "1 - 1000" then the current balance of the account must be 5000. For each account, your code should print the account id, followed by "Account state is consistent state" or "Account state is inconsistent".

(15 marks)

[CSE 105 | L-1/T-2 | 2014–15]

Q12. Write down an appropriate definition of a structure which has a `float` variable, an `int` variable, and a `char` variable. What will be the storage requirement? What will it be if you use a `union` instead? **(8 marks)**

[CSE 105 | L-1/T-2 | 2011–12]

Q13. Your friend has written the following program which gives no compiler error. However, your teacher says there is a problem. Identify it and suggest corrections:

```

1 struct ADate { int month, day, year; };
2 int main(void) {
3     struct ADate *d;
4     d->month = 3;
5     d->day   = 9;
6     d->year  = 2014;
7     return 0;
8 }
```

(6 marks)

[CSE 105 | L-1/T-2 | 2011–12]

Q14. What is the `typedef` declaration? Give suitable examples. **(5 marks)**

[CSE 105 | L-1/T-2 | 2010–11]

Q15. Write down the following declarations:

- Define a structure consisting of two floating-point members called `real` and `imaginary`, with the tag `complex`.
- Declare a variable `x` of type `complex` and assign initial values 1.3 and -2.2 to `x.real` and `x.imaginary` respectively.
- Declare a pointer variable `px` which points to a structure of type `complex`. Write expressions for the structure members in terms of the pointer variable.

(13 marks)

[CSE 101 | L-1/T-2 | 2009–10]

Q16. Write a C program to solve the quadratic equation $ax^2 + bx + c = 0$ using the `complex` structure defined above. Display both real and imaginary roots and include all possible exceptions. **(10 marks)**

[CSE 101 | L-1/T-2 | 2009–10]

Q17. Define a self-referential structure containing the following members:

- (a) A 40-element character array `name` and a 15-element character array `std_id`.
- (b) A date-type member `DOB` containing `day` (1–31), `month` (1–12), and `year` (1900–2100).
- (c) Variables `m_mark` and `p_mark` for marks (0–100) in math and physics respectively; `m_total` holds their sum.
- (d) A pointer `next` to another structure of this same type.

(10 marks)

[CSE 101 | L-1/T-2 | 2008–09]

Q18. Suppose that the Chelsea football club has 30 players. Each player has several attributes: name, date of birth, country of citizenship, number of international appearances, salary (US\$ per week), and transfer fee from the previous club. Write C program code (not a complete program) to:

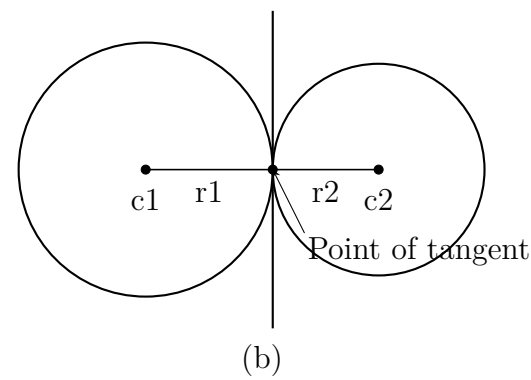
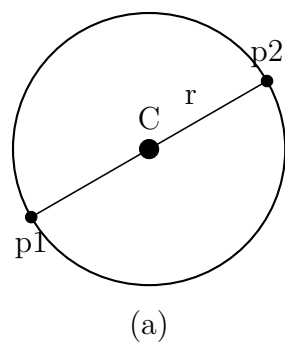
- (a) Define structure(s) to store the information of these 30 players.
- (b) Take input from the user to fill up these structures.
- (c) Search and display the name of the most valuable player (in terms of transfer fees).
- (d) Search and display the name of the youngest player.

(20 marks)

[CSE 105 | L-1/T-2 | 2005–06]

Q19. In geometry, a point is located by two coordinates (x, y) and a circle is defined by its centre (a point) and a radius. Define two structures `Point` and `Circle`, and implement the following functions:

- (a) `Circle makeCircle(Point p1, Point p2)` — returns the circle where `p1` and `p2` are the endpoints of a diameter.
- (b) `Point tangentPoint(Circle c1, Circle c2)` — returns the point of tangency of two externally tangent circles.



(13 marks)

[CSE 101N | L-1/T-1 | 2003–04]

Q20. Show the output of the following code fragment:

```

1  struct test {
2      int x;
3      int *y;
4      int **z;
5      struct test *p;
6  };
7  struct test bob;
8  bob.x = 0;
9  bob.y = &bob.x;
10 bob.z = &bob.y;
11 bob.p = &bob;
12 *bob.p->y = 1;
13 **bob.p->z = 2;
14 printf("%d", bob.x);
    
```

(6 marks)

[CSE 101N | L-1/T-1 | 2003–04]

Unions, Bit-fields & Enumerations

- Q1.** What is union data type? What is its role in memory conservation? Demonstrate the use of union and enumeration when the size of a T shirt is defined by strings like "S" (for small), "M" (for Medium), "L" (for large), "XL" (for extra Large) or the size of neck expressed by a fractional number. Illustrate the measures to be taken if you give input other than defined sizes. (10 marks) [CSE 101 | L-1/T-1 | 2024–25]
- Q2.** Explain the advantage of using enumeration data type and typedef in writing programs, with necessary examples. (8 marks) [CSE 101 | L-1/T-1 | 2024–25]
- Q3.** "Enumeration is used to save memory usage by computer programs" — justify this claim with necessary examples. (8 marks) [CSE 101 | L-1/T-1 | 2023–24]

Q4. What is the union data type? What is its role in memory conservation? Demonstrate the use of a **union** when a student's result is identified by either a letter grade **or** a percentage of obtained marks. (10 marks) [CSE 101 | L-1/T-1 | 2023–24]

Q5. Write the output of the following program:

```
1 // sizeof: long double=16, float=4, int=4, char=1, char*=8
2 typedef union { int i; char *ptr; float f; } example1;
3 typedef struct { char a; long double b; char *c; } example2;
4 typedef struct { long double b; char a; char *c; } example3;
5 typedef struct { char a[8]; int c; char b[8]; } example4;
6 int main() {
7     printf("%ld\n", sizeof(example1));
8     printf("%ld\n", sizeof(example2));
9     printf("%ld\n", sizeof(example3));
10    printf("%ld\n", sizeof(example4));
11 }
```

(10 marks)

[CSE 101 | L-1/T-1 | 2021–22]

Q6. Write the output of the following code snippet. Assume memory sharing starts from the least-significant bit.

```
1 union test {
2     unsigned int value : 3;
3     unsigned int value2 : 4;
4 };
5 int main() {
6     union test t;
7     printf("Sizeof(t) : %d\n", sizeof(t));
8     t.value2 = 5;
9     printf("t.value : %d, t.value2 : %d\n", t.value, t.value2);
10    t.value2 = 9;
11    printf("t.value : %d, t.value2 : %d\n", t.value, t.value2);
12    t.value2 = 13;
13    printf("t.value : %d, t.value2 : %d\n", t.value, t.value2);
14 }
```

(15 marks)

[CSE 101 | L-1/T-1 | 2020–21]

Q7. BUET CSE identifies each student either by their nickname (a string of at most 10 characters) or by their student ID (an integer), but not both. Declare a data structure that allows storing this information for 120 students. Briefly explain why your chosen data structure is appropriate. (5 marks) [CSE 101 | L-1/T-1 | 2020–21]

Q8. What is printed by the following program?

```
1 struct test { unsigned a:1, b:2, c:3; };
2 int main() {
3     int i;
4     struct test x;
5     for (i = 0; i < 8; ++i) {
6         x.a = x.b = x.c = i;
7         printf("%d %d %d\n", x.a, x.b, x.c);
8     }
9 }
```

What happens if you replace `x.a = x.b = x.c = i;` with `x.c = x.b = x.a = i;`? (10 marks) [CSE 101 | L-1/T-1 | 2018–19]

Q9. Define a bit-field structure called **Flags** consisting of: **flag_1** (4 bits), **flag_2** (2 bits), and **flag_3** (2 bits). Write code to read values for those flags from the console. (5 marks) [CSE 105 | L-1/T-2 | 2014–15]

Q10. Write a structure definition called **date** containing three bit-fields: **day** (5 bits), **month** (4 bits), and **year** (7 bits). Also write a union **identity** in which a **date** variable named **birthdate** shares the same memory location with an **int** variable called **slnumber**. Use **typedef** if necessary. How many decimal digits can **year** represent? Write a C function that reads **birthdate** as an integer, extracts **day**, **month**, and **year**, and prints the date in the form **dd-mm-yy**. (15 marks) [CSE 105 | L-1/T-2 | 2012–13]

Q11. Discuss the use of a tagged union. Suppose you need to handle two types of students: one type receives marks in percentage (e.g. 90%, 89.9%), the other receives letter grades (e.g. A, A+). Use a tagged union to write a program that takes input for 5 students and displays their data. (15 marks) [CSE 105 | L-1/T-2 | 2011–12]

Q12. Deduce the storage requirement of the variable **flag** below. State the range of values allowed for **f1** and **f2**:

```
1 struct Flags {
```

```

2   int          f1:3;
3   unsigned int f2:1;
4   unsigned int f3:30;
5 } flag;

```

(6 marks)

[CSE 105 | L-1/T-2 | 2011–12]

Q13. Write a structure definition for the following situation: Define three bit-fields called **x**, **y**, and **z**, whose widths are 6 bits, 4 bits, and 6 bits respectively. Force **y** to begin at the start of a second 16-bit word. Separate **y** and **z** with 2 vacant bits. Declare a structure variable **v** and assign initial values 3, 5, and 7 to the three bit-fields. Show the graphical memory representation after assignment.

(10 marks)

[CSE 101 | L-1/T-2 | 2009–10]

Q14. Define an enumeration type called **money** with the following members and values: **penny=1**, **nickel=5**, **dime=10**, **quarter=25**, **half=50**, **dollar=100**. Declare an enumeration variable **coins** of type **money** and assign it the initial value **dime**. (5 marks) [CSE 101 | L-1/T-2 | 2009–10]

Q15. The data received from a network stream is stored in a byte (**unsigned char**) array, which can be interpreted as a list of variable-length records. The first byte of each record gives the data type as an ASCII character: **'C'** (char), **'S'** (short int), **'L'** (long int), **'F'** (float), **'D'** (double), or **'X'** (end of stream). The following bytes store a data item of that type. Provide definitions for:

(a) `void printStreamPtr(unsigned char *stream);` — print each data item on a separate line using a **void *** pointer.

(b) `void printStreamUnion(unsigned char *stream);` — print each data item on a separate line using a **struct** or **union** pointer.

(16 marks)

[CSE 101 | L-1/T-2 | 2008–09]

Q16. What is the basic difference between a structure and a union? (5 marks)

[CSE 105 | L-1/T-2 | 2005–06]

Q17. What are the advantages of using enumeration variables over using **#define** within a program? Summarise the rules for assigning numerical values to enumeration constants with a suitable example. (5 marks) [CSE 101N | L-1/T-1 | 2003–04]

BUET · CSE 101 · Structured Programming

Part IV

I/O, Bitwise, Preprocessors & Miscellaneous

I/O, Bitwise, Preprocessors & Miscellaneous

S7 — Input/Output

Console & Formatted I/O

Q1. Explain the file opening modes "r", "w", "a", "r+", "w+", and "a+". (6 marks) [CSE 101 | L-1/T-1 | 2021–22]

Q2. Some program codes are given below:

```
1 int a = 0x7c5;
2 float x = 13.25;
3 printf("%-07X %#7g %10.2e", a, x, x);
```

Show the output generated by the `printf()` call. Also show the bit-stream stored in memory when storing 13.25 using the 4-byte IEEE 754 floating-point representation. Show your working. (13 marks) [CSE 101 | L-1/T-1 | 2020–21]

Q3. A C program contains the following variable declarations:

```
1 float a = 2.5, b = 0.0005, c = 3000;
```

Show and explain the output of:

```
1 printf("%8.3f %-3e %#8f", a, b, c);
```

(6 marks)

[CSE 101 | L-1/T-2 | 2008–09]

Q4. The date of September 5, 1979 is written in different forms: 05/09/1979, 05-09-1979, and 05.09.1979. Write C code that can take a date as input in any of these three forms using a single `scanf()` function call. (5 marks) [CSE 105 | L-1/T-2 | 2005–06]

Q5. Write down a `scanf()` function call statement that takes a string terminated by a newline character as input. (5 marks) [CSE 105 | L-1/T-2 | 2005–06]

Q6. Explain the following statements where `var` is a character array containing 50 elements:

- (a) `scanf("%s", var);`
- (b) `scanf("%s", &var);`
- (c) `scanf("%s", &var[0]);`

(9 marks)

[CSE 105 | L-1/T-2 | 2005–06]

File I/O & Command Line Arguments

Q1. Explain the necessity of the file opening modes specified by "r+", "w+" and "a+" by briefly describing the scenarios. (6 marks) [CSE 101 | L-1/T-1 | 2024–25]

Q2. Read line by line from two text files and check whether they match. Stop when you get the first mismatch and display the two mismatched lines. (10 marks) [CSE 101 | L-1/T-1 | 2024–25]

Q3. Explain the file opening modes "r", "w", "a", "r+", "w+", and "a+" in C's `fopen`. (6 marks) [CSE 101 | L-1/T-1 | 2023–24]

Q4. You are to manage the marks of m subjects for a number of students using a file. Write a program that:

- (a) Takes m subject marks and a student ID, and stores them in a file (data persists after program exit).
- (b) Given a student ID, retrieves and displays that student's marks.
- (c) Outputs the total marks for each student and the highest mark for each subject.

(13 marks)

[CSE 101 | L-1/T-1 | 2023–24]

Q5. Develop a C program that retrieves two numbers from command line arguments, calculates their sum, and displays the result. You may use any library function. (12 marks) [CSE 101 | L-1/T-1 | 2022–23]

Q6. Suppose you have a file containing the following contents (one integer per line): 185, 170, 165, 178, 167, 137, 156. Write a C program to read this file and sum up all the **odd** numbers. The file path is passed as a command line argument. (11 marks) [CSE 101 | L-1/T-1 | 2022–23]

Q7. The following code reads 10 integers from a file and writes their sum at the end. Make any necessary corrections:


```

1 FILE *fp = fopen("input.txt", "r");
2 int num = 0, sum = 0;
3 for (int i = 0; i < 10; i++) {
4     fscanf(fp, "%d", &num);
5     sum += num;
6 }
7 fprintf(fp, "%d", sum);

```

(4 marks)

[CSE 101 | L-1/T-1 | 2021–22]

Q8. Write a program that creates a new file containing the contents of a given text file written twice. If the input file contains "Dhaka is the capital of Bangladesh", the output file should contain that line twice. (10 marks) [CSE 101 | L-1/T-1 | 2021–22]

Q9. Take marks for 10 students in 5 subjects as input. Find and display the names and total marks of the students with the highest and lowest totals. You are not allowed to use arrays to store marks or names — use files instead. (15 marks) [CSE 101 | L-1/T-1 | 2021–22]

Q10. Read from `in.txt` until EOF. Each line contains a string and a number. For each line, determine (i) whether the string is a palindrome, and (ii) whether the number is odd or even. Write one output line per input line to `out.txt`.

Example in.txt: ABED 21 / ABSBA 23 / NIBIR 26

Example out.txt: Not-Palindrome Odd / Palindrome Odd / Not-Palindrome Even (15 marks) [CSE 101 | L-1/T-1 | 2020–21]

Q11. Define a structure `District` to store year-long weather data about a district of Bangladesh. It should contain:

```

1 Id (Integer identifier of district, between 1 and 64),
2 Temperature (An array of 365 real numbers),
3 Humidity (array of 365 real numbers),
4 Rain (An array of 365 real numbers).

```

Declare an array named `districts` of type `District`.

A file `data.txt` contains daily weather data in the format: `<district_id> <day> <temperature> <humidity> <rain>`. Information for 365 days is available for all 64 districts (not necessarily in order). Process all lines and for each weather attribute (temperature, humidity, rain), create a file named after the attribute. Each line of that file should contain a district's ID followed by its average attribute value across 365 days (district #1 first, district #64 last). (20 marks) [CSE 101 | L-1/T-1 | 2019–20]

Q12. Write a program to copy the content of a file to another file using command-line arguments for the source and destination filenames. Handle errors related to file operations and command-line argument processing. (10 marks) [CSE 101 | L-1/T-1 | 2019–20]

Q13. A binary file represents an image: the first 2 integers are height h and width w ; then $h \times w$ pixel values in RGBA model (row-major order). Some pixels are corrupted. Read from console: the image filename and the number of erroneous pixels. For each erroneous pixel, read its row, column (0-based), and the correct value. Use random access to update only the faulty pixels (do not unnecessarily read/write other pixels). (20 marks) [CSE 101 | L-1/T-1 | 2019–20]

Q14. Write a program that clones itself in reverse: copy the source file's content in reverse order and save the copy as `rcopy.c`. (10 marks) [CSE 101 | L-1/T-1 | 2018–19]

Q15. Write a C program to find the length of a file in bytes without reading the whole file. (6 marks) [CSE 101 | L-1/T-1 | 2015–16]

Q16. Write a C program that takes a series of integer inputs from the command line and prints the maximum and minimum. For example: `5c.exe 3 7 9 -1 5 7 9 10` prints 10 as maximum and -1 as minimum. (7 marks) [CSE 101 | L-1/T-1 | 2015–16]

Q17. Using the `AccountBalance` structure, write a program that reads an account ID as a command-line argument (as an integer). (You do not need to rewrite the balance reading/printing code.) (5 marks) [CSE 105 | L-1/T-2 | 2014–15]

Q18. Write a C program that accepts a file name as a command-line argument and prints the contents of the file to a printer. If no file name is provided on the command line, the program asks for the file name from standard input, then prints the file's contents to a printer. (15 marks) [CSE 105 | L-1/T-2 | 2013–14]

Q19. Write a C program that accepts two filenames as command line parameters and appends the second file to the first. If the user forgets to provide one or both filenames while running the program from the command line, the program will ask for and read the filename(s) accordingly. (15 marks) [CSE 105 | L-1/T-2 | 2012–13]

Q20. Discuss the formats and differences between `read()` and `fread()` functions. (5 marks) [CSE 105 | L-1/T-2 | 2012–13]

Q21. Explain the purpose and semantic meaning of the following C function using necessary figures:

```

1 int main(int argc, char **argv) {
2     while (--argc > 0 && (++argv)[0] == '-')

```



```

3     while (c = *++argv[0])
4         printf("%c\n", c);
5 }

```

(10 marks)

[CSE 101 | L-1/T-2 | 2012–13, 2009–10, 2008–09]

Q22. Write a program named `myCopy` that copies the contents of one file to another. Command-line behaviour: two filenames → first is source, second is destination; one filename → it is the source, prompt for destination; no filenames → prompt for both. In case of any error opening either file, inform the user and ask for another filename. (22 marks) [CSE 105 | L-1/T-2 | 2011–12]

Q23. You want to open an existing file for both read and write operations. Your friend says you can use `'r+'`, `'w+'`, or `'a+'` as the mode. Do you agree? Justify your answer. (7 marks) [CSE 105 | L-1/T-2 | 2011–12]

Q24. Write a program that takes a file name as input and counts the total number of lines in the file. Lines end with a newline character. The program should handle errors such as a file not found. (12 marks) [CSE 105 | L-1/T-2 | 2010–11]

Q25. Write a program that takes a number of command-line arguments (all integers) and finds the median and mode. Sort the numbers to find the median (middle element; average of two middle elements if even count). The mode is the most frequently occurring number; print "No Mode" if no number appears more than once.
Sample: {4,1,1,10,3,5,7} → median=4, mode=1; {2,1,4,7} → median=3, mode=No Mode. (20 marks) [CSE 105 | L-1/T-2 | 2010–11]

Q26. (a) Complete the following structure definition by declaring fields for book title (at most 99 chars), author name (at most 49 chars), and price (a real number):

```

1 struct Bookinfo {
2     /* title, author, price */
3 };

```

(b) Using the above structure, write a `main` program that reads info for 10 books and stores them in `"bookinfo.bin"` using `fwrite`.

(c) Write another program that asks for a book title, searches `"bookinfo.bin"` using `fread`, and prints the price. Print "Sorry, we do not have the book" if not found.

(25 marks)

[CSE 105 | L-1/T-2 | 2010–11]

Q27. A data structure has fields: `name` (80 chars), `address` (100 chars), and `date_of_birth` of a type defined as:

```

1 typedef struct {
2     unsigned month : 4;
3     unsigned day   : 5;
4     unsigned year  : 7;
5 } date;

```

The file `"data.old"` contains records. Write a C function that creates a new file `"data.new"` and copies all information from `"data.old"`, transferring one person's data at a time. (10 marks) [CSE 101 | L-1/T-2 | 2009–10]

Q28. The skeletal outline of a C program is shown. Write the necessary program code to: (i) read the string `name` from `sample.old`; (ii) display it on screen; (iii) accept an updated string; (iv) write the new string to `sample.new`.

```

1 #include <stdio.h>
2 main() {
3     FILE *pt1, *pt2;
4     char name[80];
5     pt1 = fopen("sample.old", "r");
6     pt2 = fopen("sample.new", "w");
7     /* ... your code ... */
8     fclose(pt1); fclose(pt2);
9 }

```

(12 marks)

[CSE 101 | L-1/T-2 | 2009–10]

Q29. Write a program that takes one or more command-line options followed by a keyword and performs the following:

- (a) `-l` : print the keyword in lower case
- (b) `-u` : print the keyword in upper case
- (c) `-t` : toggle the case of each alphabetic character in the keyword
- (d) `-n` : print the number of characters in the keyword

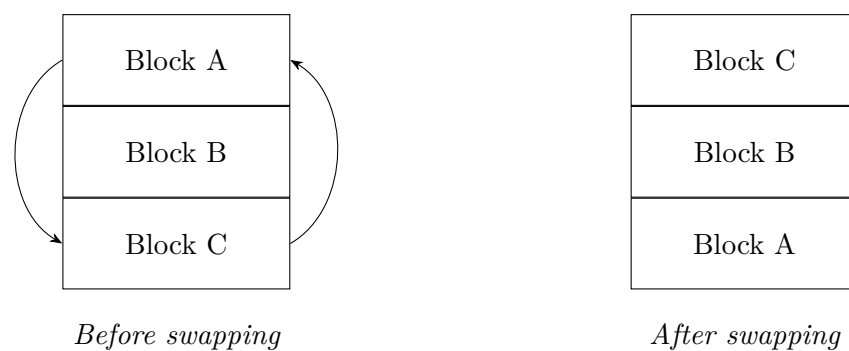
Options start with `-`. Multiple options may be grouped (e.g. `-uln` is equivalent to `-u -l -n`). Access command line arguments using pointer operations. (17 marks) [CSE 105 | L-1/T-2 | 2007–08]

- Q30.** Write a function prototyped as `void concat_file(char f1[], char f2[], char f3[])` which concatenates files `f1` and `f2` into file `f3`. **(10 marks)** [CSE 105 | L-1/T-2 | 2007–08]
- Q31.** What are the differences between a text file and a binary file? Write a function that finds the size of a file. **(15 marks)** [CSE 105 | L-1/T-2 | 2007–08]
- Q32.** The popular DX-BALL game shows high scores for the top 10 scorers in sorted order. Implement a similar system using a random-access file. Define an appropriate structure to store high scores (player name and score). Take 10 high scores as input, sort them in non-descending order, and store the sorted records in a random-access file. **(20 marks)** [CSE 105 | L-1/T-2 | 2005–06]
- Q33.** Write C code that opens a file "a.txt" in read mode and another file "b.txt" in write mode, then copies only the vowel letters from "a.txt" to "b.txt". **(10 marks)** [CSE 105 | L-1/T-2 | 2005–06]
- Q34.** Write a program that compares two text files, printing the **first line** where they differ. **(10 marks)** [CSE 101N | L-1/T-1 | 2003–04]
- Q35.** What are the differences between text files and binary files? Which is more efficient for storing data and why? **(5 marks)** [CSE 101N | L-1/T-1 | 2003–04]
- Q36.** Write a program that reads data from a text file "employee.txt" where each line has the format:
Each line will contain the name of an employee, followed by a tab, followed by the number of hours worked, followed by a tab, followed by the hourly pay rate, followed by a newline. For example, the file looks like the following:

```
1 Zahed Khan<tab>45<tab>20.00<newline>
2 Abdur Rahman<tab>32<tab>30.50<newline>
```

For each employee, compute and print their total pay (hours $\times$ rate). Handle the case of overtime (hours $>$ 40) if applicable. **(8 marks)** [CSE 101N | L-1/T-1 | 2003–04]

- Q37.** What are the four parameters of the `fread()` function? Explain each. **(4 marks)** [CSE 101N | L-1/T-1 | 2003–04]
- Q38.** An anonymous hacker wants to garble a file "exam-ques101.doc" (size $>$ 200 bytes) by swapping its first 100 bytes with its last 100 bytes. Write a program fragment that performs this task using `fseek`, `fread`, and `fwrite`.



Note: Both Block A and Block C are of size 100 bytes.
Block B remains unchanged.

(10 marks)

[CSE 101N | L-1/T-1 | 2003–04]

S8 — Bitwise Operations

Bitwise Operators & Applications

- Q1.** Write down a function that will determine whether a number x given as parameters can be expressed as 2^n or not, for any integer $n \geq 0$. Function shall return 1 if so, and shall return zero otherwise. You cannot use any library function to solve the problem. **(10 marks)** [CSE 101 | L-1/T-1 | 2024–25]
- Q2.** Write the following four functions using bitwise operators. You must maintain the restrictions mentioned in the 'restriction' column in the table.

Function prototype	Description	Restrictions
<code>int isEqual(int x, int y);</code>	Compares <code>x</code> and <code>y</code> for inequality. It should return 1 if the tested condition holds and 0 otherwise.	You cannot use <code>if-else</code> , logical, relational or ternary operator. You can only use bitwise operators.
<code>int copyLSB(int x);</code>	Set all bits of <code>x</code> same to least significant bit of <code>x</code>	You cannot use <code>if-else</code> , logical, relational or ternary operator. You can only use bitwise operators.
<code>int bitCount(int x);</code>	Returns a count of the number of 1's in the argument.	No restrictions.
<code>int xtractRight(int x, int n);</code>	Returns rightmost <code>n</code> bits of the integer <code>x</code> . Assume $1 \leq n \leq 32$	No restrictions.

(20 marks)

[CSE 101 | L-1/T-1 | 2024–25]

Q3. Consider the following program fragment which was written to find second largest of three numbers given as input. The variable `secondL` is supposed to hold the second largest of three numbers. Complete the statement in Line 7 below using only bitwise operators so that it correctly assigns second largest of three numbers to `s`.

```
1 int a, b, c, max, min, s;
2 scanf("%d%d%d", &a, &b, &c);
3 max = (a > b ? a : b);
4 max = (max < c ? c : max);
5 min = (a < b ? a : b);
6 min = (min > c ? c : min);
7 secondL =
8 printf("Second Largest", s);
```

(5 marks)

[CSE 101 | L-1/T-1 | 2024–25]

Q4. Write a function `int allRepeated(int x)` that returns 1 if and only if the positive integer `x` consists **only** of pairs of repeated digits, and 0 otherwise. You **must** use the XOR operator (5 marks deducted otherwise). Examples:

Function call	Return
<code>allRepeated(7)</code>	0
<code>allRepeated(77)</code>	1
<code>allRepeated(7733)</code>	1
<code>allRepeated(777)</code>	0
<code>allRepeated(7777)</code>	1
<code>allRepeated(2323)</code>	0
<code>allRepeated(22332)</code>	0

(20/15 marks)

[CSE 101 | L-1/T-1 | 2023–24]

[CSE 105 | L-1/T-2 | 2012–13]

Q5. Write a function `int countMinBits(int n)` that counts and returns the minimum number of bits required to represent the integer `n`. You **must** use bitwise operators. (15 marks)

[CSE 101 | L-1/T-1 | 2023–24]

Q6. Write a function `int findTrailingZeros(int n)` that finds and returns the total number of trailing zeros in the binary representation of the integer `n`. You **must** use bitwise operators. (15 marks)

[CSE 101 | L-1/T-1 | 2023–24]

Q7. Write the following four functions in C conforming to the restrictions listed beside each:

Function Name	Description	Restrictions
<code>negate(x)</code>	returns <code>-x</code>	Legal operators: <code>~ +</code>
<code>isEqual(x,y)</code>	Compares <code>x</code> to <code>y</code> for inequality. It should return 1 if the tested condition holds and 0 otherwise.	Legal operators: <code>! ~ & ^ << >></code> You cannot use any loop/recursion.
<code>copyLSB(x)</code>	Replicates a copy of the least significant bit in all 32 bits of the result.	Legal operators: <code>! ~ & ^ << >></code> You cannot use any loop/recursion.
<code>isPositive(x)</code>	Return 1 if <code>x > 0</code> , return 0 otherwise	Legal operators: <code>! ~ & ^ << >></code> You cannot use any loop/recursion.

(16 marks)

[CSE 101 | L-1/T-1 | 2022–23]

Q8. Write a C program that inputs two integers and determines how many bits they have **in common** (same bit value at the same position). For instance, -1 and 1 have exactly one bit in common; 201 and 32767 have 21 bits in common.

```
1 +1 =0000 0000 0000 0000 0000 0000 0000 0001
2 -1 =1111 1111 1111 1111 1111 1111 1111 1111
```

```
1 201=0000 0000 0000 0000 0000 0000 1100 1001
2 32767=0000 0000 0000 0000 0111 1111 1111 1111
```

(12 marks)

[CSE 101 | L-1/T-1 | 2022–23]

Q9. Answer the questions below for the given program (Listing 8):

```
1 #include <stdio.h>
2
3 // print the bits of integer x (assume little endian machine)
4 void printBits(int x);
5
6 // set the bit at position p
7 int setBit(int x, int p);
8
9 // clear the bit at position p
10 int clearBit(int x, int p);
11
12 // invert the bit at position p
13 int invertBit(int x, int p);
14
15 // set left n bits starting from position p
16 int setBits(int x, int p, int n);
17
18 // clear left n bits starting from position p
19 int clearBits(int x, int p, int n);
20
21 // invert left n bits starting from position p
22 int invertBits(int x, int p, int n);
23
24 // right circular shift by n bits
25 int rightCircularShift(int x, int n);
26
27 // left circular shift by n bits
28 int leftCircularShift(int x, int n);
29
30 int setDefaultSystemStatus(int x)
31 {
32     // x has a total of 32 bits
33     // the first 16 bits are used for storing the os status
34     // the next 12 bits are used for storing the total number of running processes
35     // the next 3 bits are used for the error status
36     // the last bit is unused
37
38     // setting the os status to 0xff11 (1111111100010001)
39     x = clearBits(x, 0, 16);
40     x = setBit(x, 0);
41     x = setBit(x, 4);
42     x = setBits(x, 8, 8);
43
44     // setting the total number of processes to 0xf43 (111101000011)
45     x = clearBits(x, 16, 12);
46     x = setBits(x, 16, 2);
47     x = setBit(x, 22);
48     x = setBits(x, 24, 4);
49
50     // setting the error status to 0x6 (110)
51     x = clearBits(x, 28, 3);
52     x = setBits(x, 29, 2);
53
54     return x;
55 }
56
57 int main()
58 {
59     int x;
60     // x has a total of 32 bits
```

```
61 // the first 16 bits are used for storing the os status
62 // the next 12 bits are used for storing the total number of running processes
63 // the next 3 bits are used for the error status
64 // the last bit is unused
65 x = setDefaultSystemStatus(x);
66 printBits(x);
67 }
```

- (a) Implement the following functions: (You cannot change any function prototypes and follow the instructions given in the comments. You must use bit manipulation techniques and cannot use any loops and cannot include any libraries).
- (i) `printBits` (4 marks)
 - (ii) `setBit`, `clearBit`, and `invertBit` (2×3=6 marks)
 - (iii) `setBits`, `clearBits`, and `invertBits` (3×3=9 marks)
 - (iv) `rightCircularShift`, and `leftCircularShift` (3+3=6 marks)
- (b) Write a new program where you must implement the functionalities of the `setDefaultSystemStatus` function using Bit Field Structure. Note that you cannot use any bit manipulation technique in this program. (10 marks)

[CSE 101 | L-1/T-1 | 2021–22]

Q10. Upon getting admission into BUET, a student gets a student ID comprising 7 digits. The first two digits are used for representing year, the next two digits for dept code and the last three digits for serial number as shown below:

Year		Dept Code		Serial number		
2	0	0	5	4	5	6

Consider that the program uses student ID for interface i.e., input and output purposes but use a student code for storing it inside computer memory. The 2 digits year in student ID is converted into 4 digits in student code. For example, if the year in student ID is 20, then it is converted to 2020 in student code. 4-byte integer variables are used for representing both the student ID and the student code. The most significant 1 bit, i.e., sign bit of student code is kept unused and always contains zero. The numbers of bits used for representing Year, Dept code and Serial number in student code are 13, 9 and 9, respectively, as showing below-

Unused	Year	Dept code	Serial number
1 bit	13 bits	9 bits	9 bits

Write a C function using the following prototype to convert a student ID to a student code.

```
1 int getStdCode(int stdID);
```

Also write a C function to convert a student code into a student ID using the function prototype given below-

```
1 int getStdID(int stdCode);
```

Use bitwise operations to solve the problem along with simple multipliers (like, 10 or 100). Don't use any array, pointer or big hard-coded multiplier (say, multiplying by 512) in your functions. (16 marks)

[CSE 101 | L-1/T-1 | 2020–21]

- Q11.** Parity of a number refers to whether it contains an odd or even number of 1-bits. The number has “odd parity”, if it contains odd number of 1-bits and is “even parity” if it contains even number of 1-bits.
- Write a function with the following signature to check the parity of the binary representation of a given unsigned integer. If the argument given has “odd parity”, it returns 1, otherwise, it returns 0.

```
1 int check_parity(unsigned int n);
```

You must use bitwise operations, and cannot use any arithmetic operations on the given number *n*. (15 marks)

[CSE 101 | L-1/T-1 | 2020–21]

- Q12.** Write a function `void swap_swag(unsigned int *a, unsigned int *b)` that swaps two unsigned integers using only bitwise operations, without any temporary variable or arithmetic operations. (10 marks)
- Q13.** Write a function that takes an unsigned integer and checks whether its bit pattern forms a palindrome. For example, 01001100 is not a palindrome, but 01100110 is. (10 marks)
- Q14.** In the RGBA color model, a 32-bit integer encodes a pixel as: bits 31–24 = ALPHA, bits 23–16 = BLUE, bits 15–8 = GREEN, bits 7–0 = RED. Write a program that takes a 32-bit unsigned integer as input and outputs the RED, GREEN, BLUE, and ALPHA intensities.

<i>bit₃₁ – bit₂₄</i> <i>(higher order bits)</i>	<i>bit₂₃ – bit₁₆</i>	<i>bit₁₅ – bit₈</i>	<i>bit₇ – bit₀</i> <i>(lower order bits)</i>
ALPHA	BLUE	GREEN	RED

(20 marks)

[CSE 101 | L-1/T-1 | 2019–20]

Q15. Write the following functions using **only bitwise operators** (no logical/arithmetic/relational operators, no if-else, no loops):

- (a) `int copyABit(int x, int n)` — copies the n -th bit of x ($0 \leq n \leq 31$) to all bit positions.
- (b) `int isPositive(int x)` — returns -1 if x is positive and 0 otherwise.
- (c) `int negate(int x)` — returns $-x$.

(15 marks)

[CSE 101 | L-1/T-1 | 2018–19]

Q16. Write a function that reverses the bit representation of a 32-bit integer. For example:

10101110 11011010 01101111 11011011 $\rightarrow$ 11011011 11110110 01011011 01110101. (10 marks)

[CSE 101 | L-1/T-1 | 2018–19]

Q17. Create a function `int bitParity(int x)` that returns 1 if there is an odd number of 0s in the binary representation of x , and 0 otherwise. You must use bitwise operators. (10 marks)

[CSE 101 | L-1/T-1 | 2015–16]

Q18. A number is *sparse* if it has no two adjacent 1-bits in its binary representation (e.g. $5 = 101$ is sparse, but $6 = 110$ is not).

- (a) Write a utility function `isSparse(x)` that returns 1 if x is sparse and 0 otherwise.
- (b) In `main`, read n and find the smallest sparse number $\geq n$ using `isSparse`.

(13 marks)

[CSE 101 | L-1/T-1 | 2015–16]

Q19. Create `setbit(x,p)` (sets bit at position p of x to 1) and `resetbit(x,p)` (resets bit at p to 0). Using these, write `setbits(x,p,n)` and `resetbits(x,p,n)` that set/reset n bits starting from position p , leaving all other bits unchanged. (12 marks)

[CSE 101 | L-1/T-1 | 2015–16]

Q20. Write the following functions using **only bitwise and arithmetic operators** (no logical/relational operators, no if-else, no loops):

- (a) `int isNotEqual(int x, int y)` — returns 0 if $x = y$, nonzero otherwise.
- (b) `int copyABit(int x, int n)` — copies the n -th bit of x to all bit positions ($1 \leq n \leq 32$).
- (c) `int isNegative(int x)` — returns 0 if x is positive, 1 otherwise.
- (d) `int isSmaller(int x, int y)` — returns 0 if $x \geq y$, nonzero otherwise.
- (e) `int invertBits(int x, int p, int n)` — inverts n bits of x starting at position p .

(25 marks)

[CSE 101 | L-1/T-1 | 2015–16]

Q21. Write a function that takes a 32-bit integer x and returns the integer obtained by interchanging all pairs of consecutive bits.

```
1 //For example,
2 // x=1011 1111 1110 1010 1001 0000 1000 0001
3 //return value = 0111 1111 1101 0101 0110 0000 0100 0010
```

(10 marks)

[CSE 101 | L-1/T-1 | 2015–16]

Q22. Create a function `int bitparity(int x)` that returns 1 if there is an odd number of 0s in the binary form of x , and 0 otherwise. You must use bitwise operators. (10 marks)

[CSE 105 | L-1/T-2 | 2014–15]

Q23. Write a function `void matchbits(int x, int y)` that takes two integers and prints the position of each bit of x where a match occurs with the corresponding bit of y . Use bitwise operators. (13 marks)

[CSE 105 | L-1/T-2 | 2014–15]

Q24. Create functions `setbit(x, p)` and `resetbit(x, p)` that set/reset bit at position p of integer x leaving other bits unchanged. Using these, write `setbits(x, p, n)` and `resetbits(x, p, n)` that set/reset n bits of x starting from position p . (12 marks)

[CSE 105 | L-1/T-2 | 2014–15]

Q25. The following code was written to interchange the values of a and b :

```
1 a = a ^ b;
2 b = a ^ b;
3 a = a ^ b;
```

Will this work if a and b contain the same value? Explain with an example. (8 marks)

[CSE 105 | L-1/T-2 | 2013–14]

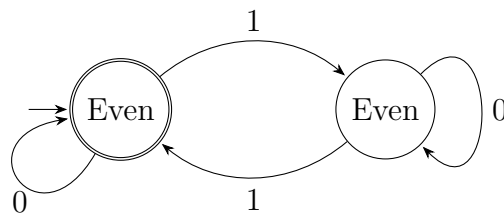
Q26. Given a set of numbers where all elements occur an even number of times except one, find the odd-occurring number using the XOR operator. Numbers are integers stored in an array of odd length n . Ask the user to enter n and all the numbers. For example, for $\{34, 52, 33, 52, 78, 78, 34, 33, 33\}$, the output is 33. (11 marks)

[CSE 105 | L-1/T-2 | 2013–14]

Q27. Write a C program that prints YES if an integer entered by the user has symmetric bits around the middle, and NO otherwise. An

n -bit integer is symmetric if bit at position i equals bit at position $n - 1 - i$ for all i . Use bitwise operators only. **(13 marks)** [CSE 105 | L-1/T-2 | 2012–13]

- Q28.** Write a C function to determine whether a bit string stored in an array `A[n]` contains an even number of 1s, using a state-transition diagram approach (automaton with two states: “Even” and “Odd” numbers of 1s). The function must follow the state transitions: the current state toggles on each 1 and stays the same on each 0. Both `A[]` and `n` are passed as arguments.

**(10 marks)**

[CSE 105 | L-1/T-2 | 2011–12]

- Q29.** Explain the bitwise right-shift operator. Write a program that takes an integer x as input and outputs how many '0's are at the end of the binary representation of $x!$ (factorial of x). Assume 64-bit integers and that $x!$ will not exceed the integer boundary. **(15 marks)** [CSE 105 | L-1/T-2 | 2010–11]

- Q30.** Give definitions for the following functions using bitwise operators:

- (a) `unsigned bw_get(unsigned x, int p, int n);` returns the right-adjusted n bits of x beginning at position p .
- (b) `unsigned bw_set(unsigned x, int p, int n, unsigned y);` returns x with the right-adjusted n bits beginning at position p set to the rightmost n bits of y , all other bits unchanged.

You may assume only valid values will be passed for p and n . **(10 marks)**

[CSE 101 | L-1/T-2 | 2008–09]

- Q31.** Write a function `int XOR(int x, int y, int p, int n)` that takes n bits of y starting at position p , performs bitwise XOR with the n bits of x starting at position p , leaves all remaining bits unchanged, and returns the result. Assume LSB is at position 0. **(10 marks)** [CSE 105 | L-1/T-2 | 2007–08]

- Q32.** In a two's-complement system, `x &= (x-1)` deletes the rightmost 1-bit in x . Explain why. Use this observation to write a function `int bitcount(int x)` that efficiently counts the number of 1-bits in x . **(10 marks)** [CSE 105 | L-1/T-2 | 2007–08]

- Q33.** Invert the middle 4 bits (bit numbers 2 to 5) of a 1-byte character variable using bitwise operators. Here, inversion means converting a bit from 0 to 1 and vice versa. **(10 marks)** [CSE 105 | L-1/T-2 | 2005–06]

- Q34.** Suppose `ch` is a `char` variable. Using bitwise operations only, determine whether the value stored in `ch` is positive or negative. **(7 marks)** [CSE 101N | L-1/T-1 | 2003–04]

- Q35.** Write appropriate bitwise expressions that will:

- (a) Place 1s in the **even** bit locations (bits 0, 2, 4, ..., 14) of a 16-bit unsigned integer v , leaving the odd bits unchanged.
- (b) Toggle (invert) bits 1 and 6 of a 16-bit unsigned integer v while preserving all remaining bits.

(5 marks)

[CSE 101N | L-1/T-1 | 2003–04]

S9 — Preprocessors, Var-Arg & Error Handling

Header Files & Preprocessors

- Q1.** Define a macro `swap(d, t, x, y)` where t is the name of a temporary variable, the values of x and y will be swapped, and d is the data type of the three variable. Use this macro in `main` to swap values of two variables. Output the values of the variables before and after swapping. **(7 marks)** [CSE 101 | L-1/T-1 | 2024–25]

- Q2.** Consider the following two definitions of the same macro:

```

1 #define square_1(x)    x*x
2 #define square_2(x)    (x)*(x)

```

Give an example where these two macros behave differently, and explain why `square_2` is probably a better definition. **(5 marks)** [CSE 101 | L-1/T-1 | 2023–24]

- Q3.** The following program contains macro-related bugs. Provide the output of the program as written, then make the necessary corrections (write only the corrected lines):

```

1 #define cube(a) a * a * a;

```

```

2  #define swap(t, x, y) t=x; x=y; y=t;
3  #define max(x, y) x > y? x: y;
4  int main() {
5      int a = 10, b = 20, t = 0;
6      if (a > b) swap(t, a, b);
7      printf("%d %d\n", a, b);
8      int c = cube(b + 2);
9      printf("%d\n", c);
10     int max_val = max(a++, b++);
11     printf("%d %d %d\n", a, b, max_val);
12     return 0;
13 }

```

(12 marks)

[CSE 101 | L-1/T-1 | 2021–22]

Q4. Suppose we have defined two macros SQ & CUB like below:

```

1  #define SQ(x) ((x)*(x))
2  #define CUB(x) (SQ(x)*(x))

```

Define a third macro $F_POW(x)$ using the above two macros in your definition that will produce:

$$\frac{x^3}{x^8}$$

(5 marks)

[CSE 101 | L-1/T-1 | 2020–21]

Q5. Find a scenario where the macro `#define SQ(x) ((x) * (x))` leads to a compilation error. (5 marks) [CSE 101 | L-1/T-1 | 2018–19]

Q6. Given macros `#define SQ(x) ((x)*(x))` and `#define CUBE(x) (SQ(x)*(x))`, define a third macro $F_POW(x)$ using these two that computes $x^{3/4}$.

(5 marks)

[CSE 101 | L-1/T-1 | 2018–19]

Q7. Write a C preprocessor macro `min3(a, b, c)` that finds the minimum of a, b, and c. Then write `min4(a, b, c, d)` that finds the minimum of four values using `min3`. (7 marks) [CSE 101 | L-1/T-1 | 2015–16]

Q8. Discuss the problems and solutions for the following parameterised macros:

- (a) `#define square(x) x*x`
- (b) `#define swap(t,x,y) t = x; x = y; y = t;`
- (c) `#define swap(t,x,y) {t = x; x = y; y = t;}`

(9 marks)

[CSE 101 | L-1/T-1 | 2015–16]

Q9. Consider the following code where the macro `abs` is intended to return the absolute value of its parameter:

```

1  #include <stdio.h>
2  #define abs(x) (x < 0 ? -x : x)
3  int x;
4  int doubleGlobalX() { x *= 2; return x; }
5  int main() {
6      scanf("%d", &x);
7      printf("%d\n", abs(doubleGlobalX()));
8      return 0;
9  }

```

What will be the output when the user inputs -5 and then 5 ? Explain your answer. (5 marks) [CSE 105 | L-1/T-2 | 2014–15]

Q10. Write a preprocessor macro `pyramid(n)` that prints a number pyramid with n rows as shown below (e.g. for $n = 3$: row 1 prints 1; row 2 prints 212; row 3 prints 32123). Why is a preprocessor faster than a function call?

For n = 3	For n = 4	For n = 5
1	1	1
212	212	212
32123	32123	32123
	4321234	4321234
		543212345

(8 marks)

[CSE 105 | L-1/T-2 | 2013–14]

Q11. Consider the two macro definitions:

```
1 #define square_1(x) x * x
2 #define square_2(x) (x)*(x)
```

Give an example where they behave differently, and explain why `square_2` is the better definition. (8 marks) [CSE 105 | L-1/T-2 | 2013–14]

Q12. Consider the macro `#define max(a, b) ((a) > (b) ? (a) : (b))`. What problem arises when executing `int z = max(i++, j++);`? Redefine the macro so that this problem does not exist. Assume the macro is applied only to integers. (8 marks) [CSE 105 | L-1/T-2 | 2013–14]

Q13. Write preprocessor directives for the following: if `flag` is 0, define `COLOR` as 1; if `flag` is less than 3 (but nonzero), define `COLOR` as 2; if `flag` is 3 or more, define `COLOR` as 3. (5 marks) [CSE 105 | L-1/T-2 | 2012–13]

Q14. Replace the following code block by defining a macro `display(i)` and using a `for` loop:

```
1 printf("x1 = %f\n", x1);
2 printf("x2 = %f\n", x2);
3 /* ... */
4 printf("x10 = %f\n", x10);
```

(5 marks) [CSE 105 | L-1/T-2 | 2012–13]

Q15. What could be the problem with the following macro definition?

```
1 #define SQUARE(x) x * x
```

(6 marks) [CSE 105 | L-1/T-2 | 2011–12]

Q16. Define a macro and write a macro in C to swap two data items. (5 marks) [CSE 105 | L-1/T-2 | 2010–11]

Q17. What are the advantages and disadvantages of using a macro over a function? (8 marks) [CSE 105 | L-1/T-2 | 2010–11]

Q18. Write the definition of a macro `CUBE(x)` that computes the cube of `x` so that the following statements all evaluate correctly:

```
1 int x, y, z;
2 x = CUBE(-5);
3 y = CUBE(4-1);
4 z = !CUBE(3);
```

(6 marks) [CSE 105 | L-1/T-2 | 2010–11]

Q19. Briefly show the uses of the following library functions with example programs:

- (a) `malloc` defined in `stdlib.h`
- (b) `atoi` defined in `stdlib.h`
- (c) `strcmp` defined in `string.h`

(9 marks) [CSE 105 | L-1/T-2 | 2010–11]

Q20. What is macro substitution? Describe its benefits. (10 marks) [CSE 105 | L-1/T-2 | 2007–08]

Q21. Write a macro `MAX(X, Y)` that finds the maximum of `X` and `Y`. Write another macro `MAX3(X, Y, Z)` that finds the maximum of `X`, `Y`, and `Z` by using `MAX(X, Y)`. (10 marks) [CSE 105 | L-1/T-2 | 2007–08]

Q22. What are the differences between a function and a macro? Write a macro for computing the maximum of 3 numbers, where these 3 numbers are given as arguments. (5 marks) [CSE 105 | L-1/T-2 | 2005–06]

Q23. Most C programs start with :

```
1 #include <stdio.h>
```

Why is it necessary? How does the compiler deal with this type of statement? (7 marks) [CSE 105 | L-1/T-2 | 2005–06]

Q24. What is the main advantage of using macros? How is a multiline macro defined in C? (5 marks) [CSE 101N | L-1/T-1 | 2003–04]

Q25. What are two different ways to define integer constants in C? (3 marks) [CSE 101N | L-1/T-1 | 2003–04]

Variable Argument Functions

- Q1.** Write a function where the first parameters is a string. The string may contain multiple specifiers "%d". The subsequent parameters will be integers as specified in the string. The function will print the string by replacing the specifiers with the corresponding integers (subsequent parameters). **(12 marks)** [CSE 101 | L-1/T-1 | 2024–25]
- Q2.** Write the required function for the given program so that the main function runs correctly. The required function takes one integer and a string as fixed arguments and then a variable number of arguments. The first argument denotes the total number of lists, and the second argument denotes the format of each list. Note all the lists have the same format. Each list element can be a character, an int, or a string. The format string consists of character 's' which denotes the string element, 'i' which denotes the integer element, and 'c' which denotes the character element. The function will return the summation of all elements of all lists. The value of the string element is the summation of the ASCII values of all character.

```

1  #include <stdio.h>
2  #include <stdarg.h>
3  #include <string.h>
4
5
6  // you cannot change the main function
7  int main() {
8      int total_sum = getTotalSum(2, "sicc", "I am confident",
9          2, 'p', 'a', "I will pass", 3, 'p', 'b');
10     // first argument "2" means 2 lists
11     // second agreement "sicc" means each list has a total
12     // of 4 elements. The first element is a string, the
13     // second element is an integer, the third element is a
14     // character, and the fourth element is a character.
15     // Then a total of 8 arguments are passed as 2 lists
16     // each having 4 elements requiring 8 arguments
17     printf("%d\n", total_sum); // prints 2737
18     total_sum = getTotalSum(2, "ic", 10, 'A', 20, 'a');
19     printf("%d\n", total_sum); // prints 192 (10 + 65 + 20 + 97)
20     return 0;
21 }
```

(10 marks)

[CSE 101 | L-1/T-1 | 2021–22]

- Q3.** Write a function `isPrime` that takes a variable number of inputs and returns the total count of prime numbers among them. The first argument specifies the total number of variable arguments. For example, `isPrime(6, 3, 7, 9, 11, 15, 20)` returns 3 because 3, 7, and 11 are prime. **(10 marks)** [CSE 101 | L-1/T-1 | 2015–16]
- Q4.** Implement a function `void printPoint(int n, ...)` that prints the integer coordinates of a point in an n -dimensional space. For example, `printPoint(2, 5, 3)` prints the 2D point (5,3), and `printPoint(3, 10, 7, 8)` prints the 3D point (10,7,8). **(10 marks)** [CSE 105 | L-1/T-2 | 2014–15]
- Q5.** The function `putInt(int var)` accepts one argument and displays the value of `var` on monitor screen. There are total three functions of similar kind that displays three data types named integer, float and character. The argument `var` contains a value to be displayed. For an instance, the program code “`putChar(char c)`” displays the value of `c` on monitor screen, where `c` is of character type. Similarly, `putFloat(float x)` will display value of `x` on monitor screen with two digits after the decimal point. Using any of the above functions, write a C functions `print(const char *str,...)` that will accept a *format* string and a number of variables, and then will display the contents of the variables on monitor screen.

Some sample input and output are given below–

Sample input	Sample output
<code>print("%d", A);</code>	12
<code>print("%c %f %d", ch, x, A);</code>	P 12.34 40
<code>print("%d %d %f %c", A, B, X, ch);</code>	23 11 45.23 D

(20 marks)

[CSE 105 | L-1/T-2 | 2013–14]

- Q6.** Write a C function with prototype `double avgPrice(const char *str, ...)` that calculates the average price of all books listed as `<name, price>` pairs in a variable argument list, terminated by NULL. For example:

```

1  avg = avgPrice("book list", "mathematics", 12.75,
2      "programming with C", 20.75,
3      "physics", 23.20, NULL);
```

(10 marks)

[CSE 105 | L-1/T-2 | 2012–13]

- Q7.** Implement `int printf(char *str, ...)` as defined in `stdio.h` using the variable argument passing technique. **(15 marks)** [CSE 101 | L-1/T-2 | 2009–10]

Error Handling

Q1. Identify all the problems in the following code segment:

```

1  int main() {
2      int a;
3      int diff;
4      int *q;
5      double *p;
6      a = 10;
7      p = &a;      /* line 8 */
8      *p = 100;    /* line 9 */
9      *q = 10;     /* line 10 */
10     p = p + 2;   /* line 11 */
11     p = p - 2;   /* line 12 */
12     p = p * 1;   /* line 13 */
13     p = p / 1;   /* line 14 */
14     diff = p - q; /* line 15 */
15     return 0;
16 }
```

(6 marks)

[CSE 101 | L-1/T-1 | 2015–16]

Q2. Write a C program to solve the quadratic equation $ax^2 + bx + c = 0$, where a , b , and c are constants taken as inputs. Include all necessary error-handling code. Output imaginary roots in the form $p + iq$ where $i^2 = -1$. (10 marks) [CSE 101 | L-1/T-2 | 2008–09]

Q3. A compiler error is reported for the following code segment. What is the possible error? Also describe the way(s) to fix it.

```

1  void test() {
2      a = 20;
3  }
4  int a;
5  void main() {}
```

(5 marks)

[CSE 105 | L-1/T-2 | 2007–08]

BUET · CSE 101 · Structured Programming

Part V

Data Structures

Data Structures

S10 — Abstract Data Structures

*Note: The following questions appeared in past CSE 105 and CSE 101 papers but fall **outside** the CSE101 syllabus. Included for reference only.*

Linked Lists

Q1. What is a doubly linked list? Define the structure of a node in a doubly linked list where each node stores two integer variables and one character variable. **(10 marks)** [CSE 105 | L-1/T-2 | 2011–12]

Q2. A doubly linked list uses a type DLL (defined via `typedef`) where each node has an integer `data`, a `next` pointer, and a `prev` pointer. The function `findNode(DLL *pHead, int target)` is given (it returns `pCur->next` if found, or `NULL`). Write the following two functions which **must** use `findNode()` to locate the target node:

- (a) `int deleteNode(DLL **pHead, int target);` — deletes the node with `data == target`; returns 1 if deleted, 0 if not found.
- (b) `int insertNode(DLL **pHead, int target, int newData);` — inserts a new node with `newData` after the node with `data == target`; returns 1 if inserted, 0 if target not found.

```

1  DLL* findNode(DLL* pHead, int target){
2      DLL* pCur;
3      //Search the nodes in a linked list
4      pCur=pHead;
5      //Search until the target value is found or the end of the list is reached
6      while(pCur!=NULL && pCur->data!=target){
7          pCur=pCur->next;
8      }
9      if(pCur!=NULL){
10         return pCur->next;
11     }else{
12         return NULL;
13     }
14 }
```

(20 marks)

[CSE 105 | L-1/T-2 | 2011–12]

Q3. Assume the doubly linked list above is sorted in ascending order of `data`. Write `insertNode1()` such that if `insertNode()` returns 0 (target not found), it finds the node with the predecessor value of `target` and inserts the new node after it. **(5 marks)** [CSE 105 | L-1/T-2 | 2011–12]

Q4. Consider the following structure for a singly linked list:

```

1  struct Node { int val; struct Node *next; };
```

- (a) Write an `insert` function that inserts elements in ascending sorted order. Prototype: `struct Node *insert(struct Node *head, int val);`. The function searches for the correct position, allocates a new node, inserts it, and returns the (possibly new) head.
- (b) Write a recursive function `void print(struct Node *elem)` that prints the linked list in reverse order.

(23 marks)

[CSE 105 | L-1/T-2 | 2010–11]

Q5. What are the advantages and disadvantages of using a linked list over arrays? **(8 marks)**

[CSE 105 | L-1/T-2 | 2010–11]

Q6. Define a self-referential structure containing the following members:

- (a) A 40-element character array `name` and a 15-element character array `std_id`.
- (b) A date-type member `DOB` containing `day` (1–31), `month` (1–12), and `year` (1900–2100).
- (c) Variables `m_mark` and `p_mark` for marks (0–100) in math and physics respectively; `m_total` holds their sum.
- (d) A pointer `next` to another structure of this same type.

Write a C function to create a linked list for storing students' information (using the self-referential structure with `name`, student ID, date of birth, physics mark, math mark, and a `next` pointer). **(15 marks)** [CSE 101 | L-1/T-2 | 2008–09]

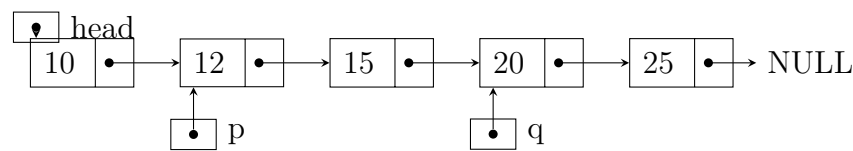
Q7. Suppose you have a one-way linked list where each node stores `name (char *)` and `age (short int)`. Write two functions:

- (a) `void ll_to_file(char *file_name, node *head);`
- (b) `node *ll_from_file(char *file_name);`

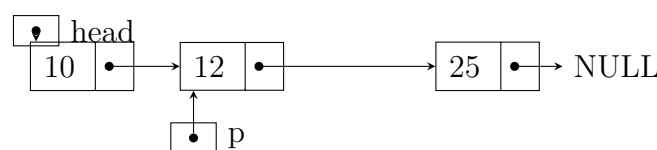
that store and retrieve the entire linked list to/from a file, preserving node order. **(25 marks)** [CSE 101 | L-1/T-2 | 2008–09]

Q8. How can you define a linked list in a C program? Compare a linked list with an array. **(10 marks)** [CSE 105 | L-1/T-2 | 2005–06]

- Q9.** Describe, with C code, the different cases that may arise during insertion of a node into a linked list. How can you insert a new node into a sorted linked list while maintaining the sorted order? **(25 marks)** [CSE 105 | L-1/T-2 | 2005–06]
- Q10.** When does a linked list have superior utility over an array? **(4 marks)** [CSE 101N | L-1/T-1 | 2003–04]
- Q11.** Write a function `void deleteNodes(Node *p, Node *q)` that deletes all nodes from just after `p` up to and including `q` from a singly linked list, adjusting the `next` pointers accordingly and freeing all deleted nodes. Assume `p` is to the left of `q` and neither is `NULL`.



Before deletion



After deletion

(10 marks)

[CSE 101N | L-1/T-1 | 2003–04]

Stacks & Queues

- Q1.** Write down a generic implementation (`init`, `push`, `pop`) of a stack that can be used with any datatype. Implement the stack as an array of `void` pointers (capacity 500). The `push` function should take the size of the data as an argument and allocate necessary memory. Use:

```
1 void *memcpy(void *dest, void *src, int n);
```

 The `pop` function should free any memory previously allocated by `push`. **(13 marks)**

[CSE 101 | L-1/T-2 | 2008–09]

- Q2.** Implement a queue for integer variables. There is an array `queue` of 100 integers. Write two functions:

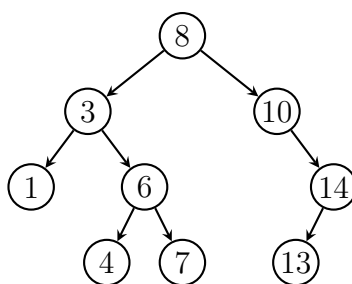
- (a) `void enqueue(int i)` — inserts `i` at the last position of existing elements.
- (b) `int dequeue()` — retrieves the front element.

 Assume other necessary variables so that the FIFO property is maintained. **(15 marks)**

[CSE 105 | L-1/T-2 | 2007–08]

Binary Search Trees

- Q1.** A tree has exactly one path from the root (e.g., the node marked with 8 in the Figure is the root) to any node. A binary tree is a tree where each node has at most two children. For example, node 3 and node 10 are the children of node 8; similarly, node 1 and node 6 are the children of node 3. Moreover, node 3 is the left child and node 10 is the right child of node 8. Also note that, nodes 1, 3, 4, 6 and 7 are in the left subtree and nodes 10, 13 and 14 are in the right subtree of node 8.



A binary search tree is a binary tree where the value of each node in the left subtree of a node is smaller than the value of that node, while the value of each node in the right subtree is equal to or higher than the value of that node. A binary search tree is represented by a 1D array as follows—

- (i) The root is placed at index 1.
- (ii) If a node is in index i , then its left child is placed at index $2i$ and the right child is placed at index $2i + 1$.
- (iii) -1 indicates the node does not exist.

Write a recursive C function that will generate a binary search tree from some given values by inserting one value at a time using the following function prototype—

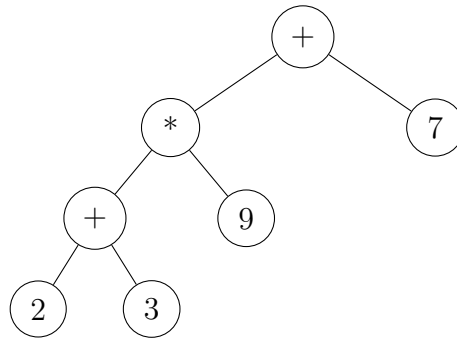
```

1  int insertNode(int *tree, int x);
2  // x will be inserted into tree as a node.

```

Also write a recursive C function that counts the number of nodes available in the tree. **(13 marks)** [CSE 101 | L-1/T-1 | 2020–21]

- Q2.** (a) A binary expression tree is a specific kind of a binary tree used to represent expressions. The leaves of the binary expression tree are operands, such as constants or variable names, and the other nodes contain operators. Assume the set of possible operators are $\{+, -, *, /\}$. The set of possible operands are $[0-9]$. See the figure below, which is the binary expression tree for the expression (in in-fix notations): $((2 + 3) * 9) + 7$



The input contains several lines to describe an expression tree, until the end of file. Each line describes a node of the tree. Such a line starts with a sequence of 'L' and 'R's, that represents the turns (left or right) you have to make to reach the node, starting from the root. The final character represents the content of the node – either an operand or an operator. It is guaranteed that a child node's description never comes ahead of its parent node. Also, there are no unary operators used in the expression. For the given figure, the input would be:

```

1  +
2  L*
3  R7
4  LL+
5  LR9
6  LLL2
7  LLR3

```

Given description of a particular binary expression tree, you need to construct it using self referential data structure (i.e. a structure definition which includes at least one member that is a pointer to the structure of its own kind). Store the root of the tree in a local variable called **root**.

Do not use global variables. (20 marks)

- (b) Postfix notation is a mathematical notation in which every operator follows all of its operands. An expression in postfix notation is parenthesis-free. For example, for the infix expression $((2 + 3) * 9) + 7$, the expression in postfix notation is: $2\ 3\ +\ 9\ *\ 7\ +$. Using the expression tree, constructed in 8(a) and represented by the variable **root**, write necessary code to produce the expression in postfix notation. There should be a space printed after printing each operator/operand. **(10 marks)**

[CSE 105 | L-1/T-2 | 2014–15]

- Q3.** A BST is stored in a 1-D array where the root is at index 1; the left and right children of **node[i]** are at **node[2*i]** and **node[2*i+1]**; value -1 means no element.

- (a) Write a recursive C function **void BST_insert(int node[], int x)** that inserts element **x** into the BST. If the tree is empty, **x** becomes the root.
- (b) Write a recursive C function that returns the height of a given node (root has height 0; a non-existent node has height -1).

(20 marks)

[CSE 105 | L-1/T-2 | 2013–14]

- Q4.** A C program uses the following BST structure:

```

1  typedef struct tt {
2      int value;
3      struct tt *lchild;
4      struct tt *rchild;
5  } node;
6  node *tree;

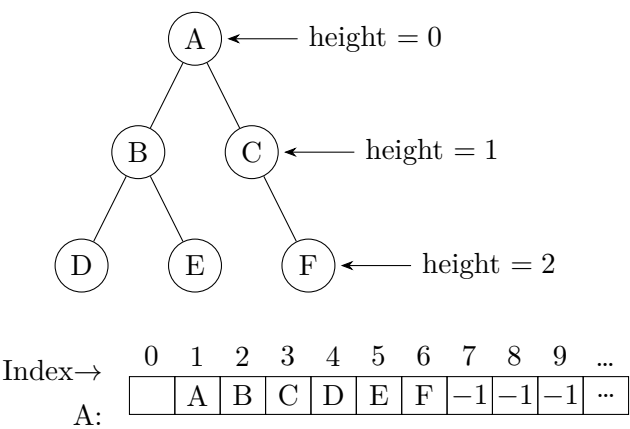
```

For each node, all values in the left subtree are less than the node's value, and all values in the right subtree are greater. Write a C function **void insertnode(node *n)** that inserts node **n** into the tree.

(15 marks)

[CSE 105 | L-1/T-2 | 2012–13]

- Q5.** A binary search tree is represented as a 1-D array where the root is at index 1. For a node at index i : its left child is at $2i$ and right child at $2i + 1$. Non-existing nodes are represented by -1 .



- (a) Write a C function that reads integers from standard input and places them in array `A[n]` following the 1-D BST representation principle. Assume `A[]` and `n` are declared globally.
- (b) Write a C function to calculate the height of an element in the tree, where the root’s height is 0. (A non-existent node has height -1 .)

(20 marks)

[CSE 101 | L-1/T-1 | 2015–16]

[CSE 105 | L-1/T-2 | 2011–12]

- Q6.** Consider a Binary Search Tree (BST) of integers. Write functions for the following tasks:
- (a) Find the number of integers in the BST that are larger than a given integer x .
- (b) Find the largest number in the BST.
- (c) Print the numbers in the BST in descending order.

(21 marks)

[CSE 101 | L-1/T-2 | 2008–09]

- Q7.** Write C code along with the necessary data structure for:
- (a) Building a Binary Search Tree (BST) of strings from consecutive input words.
- (b) Printing the BST in lexicographically descending order.

Draw a picture of the resulting BST if the strings are inserted in the following order:
“it is the best time for all good men to come to the aid of their party”. (25 marks)

[CSE 105 | L-1/T-2 | 2007–08]